

# Reliable Open-Set Network Traffic Classification

Xueman Wang<sup>ID</sup>, Yipeng Wang<sup>ID</sup>, *Senior Member, IEEE*, Yingxu Lai<sup>ID</sup>, Zhiyu Hao<sup>ID</sup>,  
and Alex X. Liu<sup>ID</sup>, *Fellow, IEEE*

**Abstract**—The widespread use of modern network communications necessitates effective resource control and management in TCP/IP networks. However, most existing network traffic classification methods are limited to labeled known classes and struggle to handle open-set scenarios, where known classes coexist with significant volumes of unknown classes of traffic. To solve this problem more accurately and reliably, we propose RoNeTC. This method achieves high-precision classification by enhancing feature extraction and quantifying the reliability of classification decisions through uncertainty estimation. For feature extraction, we divide each packet of a flow into three views for parallel training, integrating both local and global feature representations across multiple packets to enhance accuracy. We devise a second-order classification probability to quantify the reliability of the classifier's results and to visualize the reliability of open-set flow classification in terms of uncertainty. Additionally, we dynamically fuse classification decisions from multiple views, evaluating decision uncertainty to classify known and unknown flows and ensure robust, reliable results. We compare RoNeTC with four state-of-the-art (SOTA) methods in six open-set scenarios. RoNeTC outperforms the other methods by an average of 25.94% in *F1* across all open-set scenarios, indicating its superior performance in open-set network traffic classification.

**Index Terms**—Network security and privacy, open-world network traffic classification, unknown classes, deep learning.

## I. INTRODUCTION

THIS paper focuses on network traffic classification in open-set environments, which are scenarios that contain both known and unknown classes of network traffic. Network traffic classification is a task that associates network traffic with application protocols or the applications that generate the traffic. It is crucial for network security areas such as differentiated services, network intrusion detection, and resource assignment [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]. However, with the rapid development of network technology, a new challenge has emerged. This challenge arises from the mismatch between the increasing number of unknown application classes and the limitations of traditional classifiers,

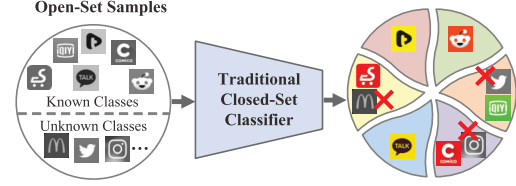


Fig. 1. The red 'x' marks indicate instances where unknown classes have been incorrectly classified as known classes.

which can only recognize known application classes. As shown in Fig. 1, in open-set scenarios containing both known and unknown classes, traditional closed-set classifiers fail to identify unknown classes of flows. Typically, they wrongly classify all unknown classes as known ones, significantly reducing the accuracy of the classification results. To address this issue, several scholars have begun initial explorations into correctly classifying known and unknown application traffic in open-set scenarios.

Open-set network traffic classification can be broadly categorized into two groups based on the type of model input: packet payload-based methods [4], [5] and packet time series-based methods [1]. The packet payload method uses raw bytes from the packet as model inputs. However, features focus solely on the byte contents within a packet, making it difficult to capture the correlation between different packets of a flow. Methods based on packet time series primarily use the total packet length from the network layer as input and focus on constructing the time series correlation of packets in a flow. However, these methods often omit crucial protocol fields, such as port number, window size, and server name indication, in the transport or application layers, which are vital for classifying network traffic.

Open-set network traffic classification can also be roughly divided into two categories according to its threshold division: methods based on classification probability thresholds [4], [5] and methods based on gradient thresholds [1]. The classification probability threshold-based uses a softmax function to output a classification probability value for each sample and divides the probabilistic threshold to classify known and unknown flow with the help of a Cumulative Distribution Function (CDF) of probability scores. The gradient threshold-based methods evaluate the feed-forward inference derived from backpropagation and classify by evaluating gradient sensitivity for known and unknown classes flows. Both methods use softmax to output the classification probability value of the flow and assign the corresponding labels based on the classification probability. However, since the softmax function tends to inflate the classification probability values due to

Received 11 June 2024; revised 7 December 2024 and 24 January 2025; accepted 10 February 2025. Date of current version 27 February 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62472011 and in part by the Beijing Natural Science Foundation under Grant L244009. The associate editor coordinating the review of this article and approving it for publication was Prof. Mika Ylianttila. (Corresponding author: Yipeng Wang.)

Xueman Wang, Yipeng Wang, and Yingxu Lai are with the College of Computer Science, Beijing University of Technology, Beijing 100124, China (e-mail: wangxueman@emails.bjut.edu.cn; yipeng.wang1@gmail.com; laiyngxu@bjut.edu.cn).

Zhiyu Hao is with the Zhongguancun Laboratory, Beijing 102629, China (e-mail: haozy@zgclab.edu.cn).

Alex X. Liu is with Midea Group, Foshan 528311, China (e-mail: alexliu360@outlook.com).

Digital Object Identifier 10.1109/TIFS.2025.3544067

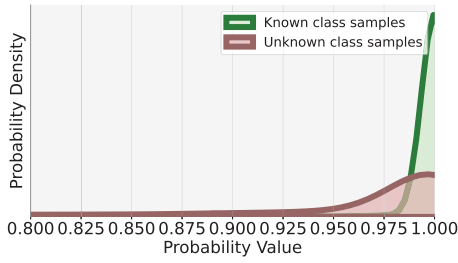


Fig. 2. Classification probability density figure for known and unknown classes output by softmax function.

its computational mechanism, leading to unreliable results [12], [13]. This issue becomes more pronounced in open-set scenarios.

As shown in Fig. 2, for the unknown class flows, the softmax function still outputs highly confident classification probability value ( $> 95\%$ ) and wrongly assigns the unknown class flows to the known class label sets, resulting in unreliable results.

In this paper, we propose RoNeTC, a **Reliable Open-Set Network Traffic Classification**. RoNeTC is based on a key insight: Due to a priori knowledge of the known classes, the classification decision output by the classifier for known class flows, which is the probability mass function of the sample, is highly certain. For unknown class flows, the probability mass function is uncertain. Therefore, we can deal with the classification problem of network traffic in an open-set environment by quantifying the reliability of the classifier's output decision. This involves enhancing the classifier's decision process by utilizing uncertainty estimates, adding a metric into the classifier's decision in the form of a second-order classification probability, and visualising the reliability of the classification decision in an intuitively probabilistic manner. For known class flows, the classifier outputs a classification probability value with a highly confident decision reliability metric, leveraging prior knowledge from training. In contrast, for unknown class flows, due to the absence of training samples and inconsistency with known class distributions, the classifier still provides a classification probability value but with a lower confidence decision metric. Using this decision reliability metric, we can accurately distinguish between known and unknown class flows and subdivide known classes.

RoNeTC aims to classify known and unknown classes in open-set scenarios while ensuring the reliability and robustness of classification decisions. To fully utilize the effective information from packets, our input includes raw bytes from the network layer, transport layer, and application layer of multiple packets in one flow. Combined with multi-view learning, each view captures global features across multiple packets and local features within a single packet. To avoid inflated classification probabilities from the softmax layer, we remove it and model the second-order classification probabilities. This approach provides trustworthy classification results with an uncertainty estimation metric for the reliability of the classification decision.

Next, to measure the reliability of classification decisions, we construct second-order classification probabilities using the Dirichlet distribution. The Dirichlet distribution is parameterized with evidence supporting the classification, forming a distribution of classification probabilities distributions and expressing decision reliability through uncertainty estimates. Finally, dynamically fuse classification decisions from multiple views. The uncertainty of each view's classification decision is used as a weight, resulting in the final reliable classification decision and the overall decision uncertainty. In open-set flow classification, we distinguish between known and unknown classes based on decision uncertainty and further subdivide the known classes of flows based on the probability value of the decision.

Our major contributions in this paper can be concluded as follows:

- We propose RoNeTC, a novel robust and reliable method for open-set network traffic classification. RoNeTC achieves reliable classification results by measuring the degree of reliability of the classification decisions and accurately distinguishing between known and unknown classes based on decision uncertainty.
- We combine multi-view and global-local feature representations to achieve fine-grained feature characterization. Reliable classification decisions are obtained by constructing second-order classification probabilities and decision uncertainty estimates. In addition, we dynamically fuse classification decisions from multiple views based on uncertainty, enhancing the reliability and robustness of open-set classification.
- We compare RoNeTC with four SOTA open-set classification methods across six open-set scenarios. The experiments show that RoNeTC outperforms the other methods in  $F1$  by an average of 25.94% in all open-set scenarios, demonstrating its superior classification capabilities.

The rest of the paper proceeds as follows. In Section II, we introduce several recent related works. In Sections III and IV, we present the technical details of each phase of RoNeTC. In Section V, we evaluate RoNeTC with real-world application datasets. We compare RoNeTC with existing methods in Section VI. Finally, we conclude our work in Section VII.

## II. RELATED WORK

Network traffic classification can be broadly divided into closed-set and open-set scenarios, depending on the application environment. Table I provides a summary of the related work.

### A. Network Traffic Classification in the Closed-Set Scenarios

In recent years, the field of network traffic classification [2], [3], [4], [5], [6], [7], [8], [9], [18], [23] in a closed-set setting has experienced rapid advancement, fueled by the proliferation of deep learning techniques. Li et al. [2] were the first to use deep learning techniques to analyze packet payloads for network traffic classification. They designed a BSNN network that consists of a recurrent neural network

TABLE I  
SUMMARY OF RELATED WORK ON NETWORK TRAFFIC CLASSIFICATION

| Method                   | Method Type    | Granularity  | Analyse Objects                     | Feature Extraction                      | Algorithm                     | Scenario   | Uncertainty            | Published Year |
|--------------------------|----------------|--------------|-------------------------------------|-----------------------------------------|-------------------------------|------------|------------------------|----------------|
| Li et al. [2]            | Deep Learning  | Packet level | Network Traffic                     | Payload Features                        | LSTM                          | Closed-Set | NO                     | 2018           |
| Liu et al. [9]           | Traditional ML | Flow level   | Encrypted Traffic                   | Message size sequence Features          | Markov Chain                  | Closed-Set | NO                     | 2018           |
| Aceto et al. [3]         | Deep Learning  | Flow level   | Mobile Encrypted Traffic            | Payload and Sequence Features           | CNN,GRU                       | Closed-Set | NO                     | 2019           |
| Aceto et al. [6]         | Deep Learning  | Flow level   | Mobile Encrypted Traffic            | Payload and Statistical Features        | SAE, LSTM, CNN                | Closed-Set | NO                     | 2019           |
| Bikmukhamedov et al. [7] | Traditional ML | Flow level   | Plaintext and Encrypted IoT Traffic | Statistical Features                    | GradBoost, SVM, MLP           | Closed-Set | NO                     | 2019           |
| Liu et al. [8]           | Deep Learning  | Flow level   | Encrypted Traffic                   | Sequence Features                       | GRU                           | Closed-Set | NO                     | 2019           |
| Zhang et al. [4]         | Deep Learning  | Flow level   | Network Traffic                     | Payload Features                        | CNN                           | Open-Set   | NO                     | 2020           |
| Yang et al. [11]         | Deep Learning  | Flow level   | Network Traffic                     | Sequence Features                       | CNN, XGBoost                  | Open-Set   | NO                     | 2021           |
| Aceto et al. [14]        | Deep Learning  | Flow level   | Encrypted Traffic                   | Payload and Sequence Features           | CNN                           | Closed-Set | NO                     | 2021           |
| Nascita et al. [15]      | Deep Learning  | Flow level   | Mobile Application Traffic          | Statistical Features                    | CNN,LSTM, GRU                 | Closed-Set | NO                     | 2021           |
| Xiao et al. [16]         | Deep Learning  | Packet level | Network Traffic                     | Payload Features                        | LSTM, GRU                     | Closed-Set | NO                     | 2021           |
| Xu et al. [5]            | Traditional ML | Flow level   | Encrypted Traffic                   | Sequence Features                       | RF,DT,GNB,KNN                 | Open-Set   | NO                     | 2022           |
| Wang et al. [10]         | Deep Learning  | Flow level   | Encrypted Traffic                   | Sequence and Statistical Features       | Tree and CNN                  | Closed-Set | NO                     | 2022           |
| Wang et al. [17]         | Deep Learning  | Packet level | Network Traffic                     | Payload Features                        | CNN                           | Closed-Set | NO                     | 2022           |
| Bovenzi et al. [18]      | Deep Learning  | Flow level   | Mobile Network Traffic              | Statistical Features                    | CNN                           | Closed-Set | NO                     | 2023           |
| Nascita et al. [19]      | Deep Learning  | Flow level   | Network Traffic                     | Statistical Features                    | CNN, GRU                      | Closed-Set | NO                     | 2023           |
| Qu et al. [20]           | Deep Learning  | Flow level   | Network Traffic                     | Sequence Features                       | Tree, Attention, Hybrid       | Closed-Set | NO                     | 2023           |
| Chen et al. [21]         | Deep Learning  | Flow level   | Mobile Application Traffic          | Statistical, Payload, Sequence Features | C4.5 Decision Tree, Attention | Closed-Set | NO                     | 2024           |
| Qian et al. [22]         | Deep Learning  | Packet level | Website and Tor Traffic             | Sequence Features                       | CNN                           | Closed-Set | NO                     | 2024           |
| Our(RoNeTC)              | Deep Learning  | Flow level   | Network Traffic                     | Payload Features                        | CNN,Attention                 | Open-Set   | Dirichlet Distribution | –              |

and an attention mechanism. Aceto et al. proposed MIMETIC [3], a multi-model deep learning solution for network traffic classification. MIMETIC enables the examination of network traffic from complementary views. Later, Aceto et al. [14] expanded on the idea of multi-modality based on deep learning and proposed DISTILLER, which combines deep learning with multi-task learning. The aim of DISTILLER is to overcome the limitations of single-task deep learning. Nascita et al. [15] pioneered the application of deep learning model interpretation in network classification by introducing the MIMETIC-ENHANCED framework. This framework aims to enhance the reliability and interpretability of network flows using interpretable artificial intelligence (XAI) techniques. Xiao et al. proposed EBSNN [16], an extended byte segment neural network to classify network traffic. EBSNN divides a packet into header segments and payload segments, which are then fed into encoders composed of the recurrent neural networks with the attention mechanism. Wang et al. [10] proposed TaTic, a two-stage early classification scheme for encrypted traffic, which quickly distinguishes between ‘easy flow’ and ‘hard flow’ by observing only the first few packets. Nascita et al. [19] employed advanced interpretable techniques to analyze flow reliability at the biflow level within a multimodal, multitask framework, thereby achieving faster and more efficient classification models.

### B. Network Traffic Classification in the Open-Set Scenarios

All of the above methods assume that there are only known class flows in the environment. However, in real applications there is a mixture of known class flows and unknown class flows. Therefore, open-set network traffic classification, which is crucial for improving network traffic management, has become a focus of attention. AutoUA [4] is an open-set flow classification method for autonomous learning. This method involves conducting flow classification on an open-set classification dataset using a classifier trained with known-class samples, and then setting a probability threshold to classify known and unknown-class. GradBP [1] is similar to AutoUA, but it differs in threshold calculation. Instead of using a probability threshold, it employs a gradient threshold. Here, the threshold is derived from the assessment of feed-forward inference through gradient backpropagation. ETC-PS

[5] is a classification method for analyzing encrypted network traffic using path signature features. It constructs traffic paths based on session packet lengths, converts these into salient structural features, and employs multi-scale path signatures alongside random forests for both closed and open-set traffic classification. However, in the open-set context, ETC-PS is limited to two coarse classifications: known and unknown.

However, all the open-set network traffic classification methods mentioned above rely on regular classification networks with softmax functions to compute classification probabilities. In the open-set scenarios, the use of softmax to calculate classification probabilities is not reliable.

### C. Discussion on Conformal Prediction

Uncertainty is defined as the measure of the reliability of classification results, unlike traditional models that only output the classification probability of a sample. Our proposed method, RoNeTC, outputs the model’s classification decision through a Dirichlet distribution, which includes the predicted value of the sample and the uncertainty. Visualize how confident the model is in the decisions it outputs by quantifying uncertainty.

Conformal Prediction is a framework designed to provide reliable and quantitative uncertainty estimates for ML and DL prediction results. Based on statistical theory, it expresses prediction uncertainty by generating a confidence interval or a confidence set instead of a single point prediction.

We demonstrate the advantages of using the Dirichlet distribution in open-set network traffic classification from the following points.

**Sample-Level Distribution Information:** The Dirichlet distribution can provide sample-level probabilistic distribution information for open-set network traffic classification, enabling finer-grained decision-making. In contrast, Conformal Prediction only offers confidence intervals, which are insufficient for detailed analysis in complex scenarios.

**Dynamic Adaptability:** Encrypted traffic often contains both plaintext and encrypted packets. By leveraging a multi-view approach to process network traffic and incorporating the Dirichlet distribution, it is possible to dynamically integrate information from different views, thereby better adapting to complex network traffic environments.

### Training Phase

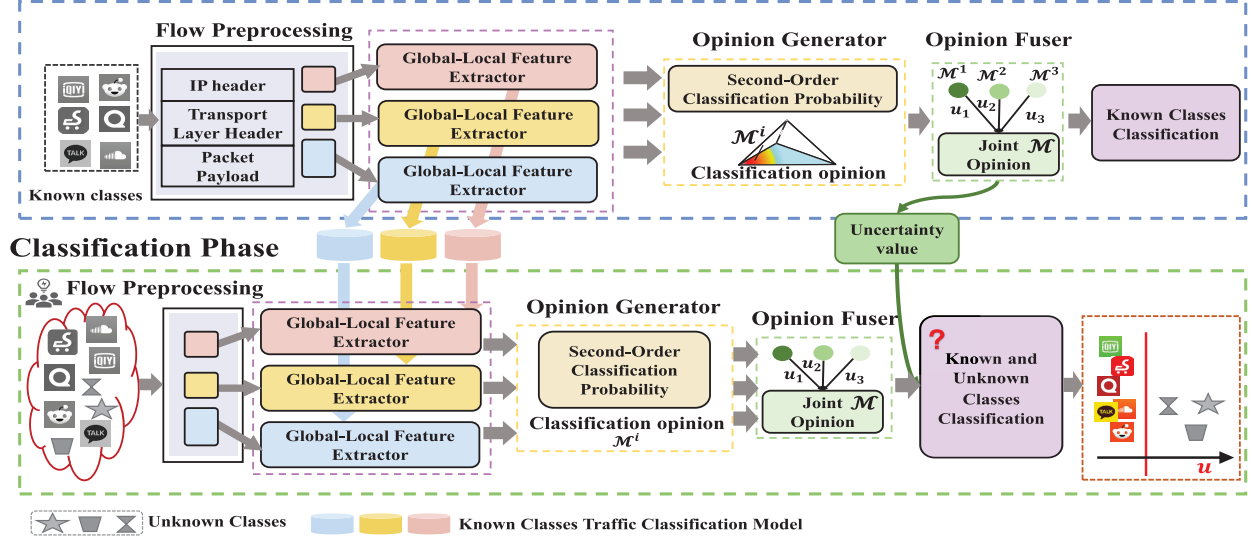


Fig. 3. Overview of RoNeTC, including training phase and classification phase.

**Handling Unknown Samples:** In open-set network traffic classification, where the quantity and distribution of unknown classes are unpredictable, the Dirichlet distribution does not rely on historical data calibration. It can efficiently adapt to changes in data distribution in real-time. In contrast, Conformal Prediction requires offline calibration, lacking the flexibility and dynamism needed for such scenarios.

### III. TRAINING PHASE OF RONECTC

Our proposed RoNeTC framework, designed for reliable network traffic classification in open-set settings, is depicted in Fig. 3. It comprises four modules: Flow Preprocessing, Global-Local Feature Extractor, Opinion Generator, and Opinion Fusion. In the training phase, since no samples of unknown class are involved, we can only rely on the prior knowledge of the known class. To obtain more reliable classification results for open-set network traffic, we design the Flow Preprocessing module to divide each packet of a bi-directional flow into three distinct views according to the TCP/IP protocol stack. Subsequently, we apply the deep learning-based feature extractor to each view, performing inter-packet and intra-packet feature refinement to achieve global-local feature extraction. Each view then outputs a classification opinion, namely, a classification decision accompanied by a measure of decision reliability termed the opinion uncertainty, generated through an opinion generator that utilizes second-order classification probabilities. Finally, the ultimate classification decision and its associated uncertainty are determined by dynamically integrating all classification opinions through the opinion fusion module, using the uncertainty of each viewpoint as a weighting factor.

#### A. Flow Preprocessing

The main objective of this subsection is to transform raw network traffic into a tensor for use in subsequent feature extractor. To construct this input tensor, we combine multiple

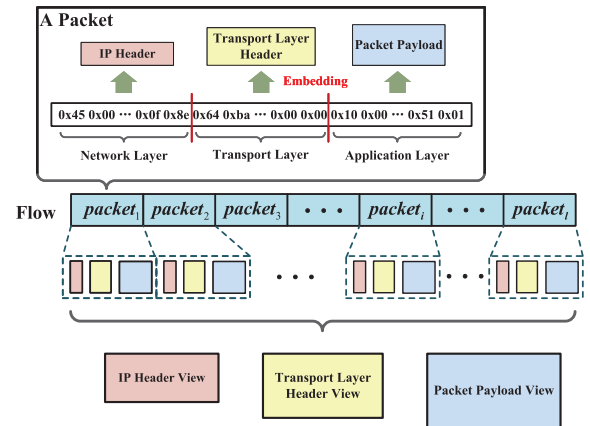


Fig. 4. A flow is divided into three parts, each represented as a separate view: IP Header, Transport Layer Header, and Packet Payload, and then encoded into a two-dimensional tensor.

views to comprehensively capture the features of known class flows. We intercept the first  $l$  packets of a bi-directional flow, where each packet shares the same 5-tuple (i.e., the same source IP address, source port, destination IP address, destination port, and the Layer 4 protocol). To more finely utilize packet characteristics, we divide each packet of a flow into three views according to the TCP/IP protocol stack, as shown in Fig. 4. In this module, the network layer is represented by the 'IP Header View', the transport layer by the 'Transport Layer Header View', and the application layer by the 'Packet Payload View'. Each packet of a flow is divided into three sets of data to form three views, and the one-dimensional bytes of each view are converted into two-dimensional tensor through the embedding layer. A flow sample is processed by the flow processing module to generate two-dimensional tensors, representing three views of multiple packets within a single flow. To prepare for the subsequent deep learning-based feature extractor, each view is trained in



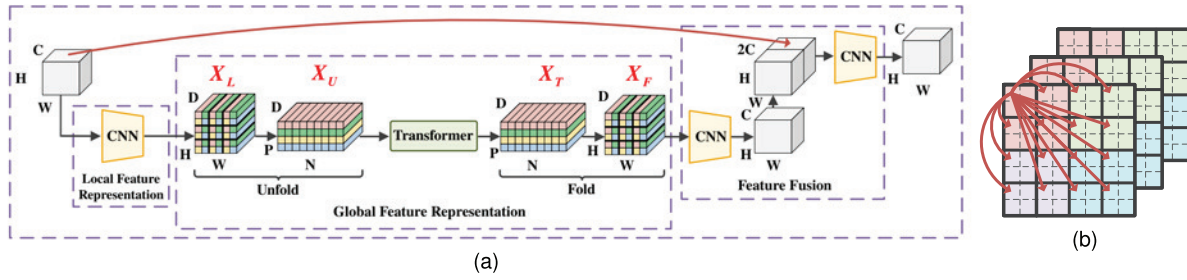


Fig. 5. (a) The Global-Local Feature Extractor. (b) The schematic of global feature representation. Each color represents a packet. Medium-sized solid grid lines indicate patches, assumed to be  $p \times p$  in size (divided by dashed lines). The Transformer operates at the corresponding positions between the patches (shown by the red curves with arrows).

parallel within the feature extractor. This design facilitates the generation of classification decisions and their corresponding reliability metrics for each view.

### B. Global-Local Feature Extractor

The Global-Local Feature Extractor is designed to extract local features within individual packets and global features across multiple packets. This extractor, which combines CNN and Transformer techniques, is illustrated in Fig. 5(a). In contrast to the weak correlation observed within single packets of a flow sequence, the inter-packet correlation proves to be strong and significant [17]. This characteristic sets our approach apart from prior methods [8], [16], which only account for local information from a single packet. It allows us to fully extract flow characteristics from both intra- and inter-packet aspects. Therefore, we designed a global-local feature extraction module to provide sufficient evidence for the subsequent modeling of second-order classification probabilities and for estimating uncertainty in classification decisions.

The Global-Local Feature Extractor is comprised of three parts: the local feature representation, the global feature representation, and the feature fusion. Specifically, for an input tensor  $X \in \mathbb{R}^{H \times W \times C}$ , where  $H$ ,  $W$ , and  $C$  are the height, width, and number of channels of the input embedding of the single view, respectively.

**Local Feature Representation:** This part is employed to encode the local feature information of a single packet in the flow sequence. It uses a standard convolutional layer with a  $3 \times 3$  receptive field, followed by a point-wise (or  $1 \times 1$ ) convolution layer that projects the tensor to a high-dimensional space ( $D$ -dimensional space, where  $D > C$ ) by learning linear combinations of the input channels to generate  $X_L \in \mathbb{R}^{H \times W \times D}$ .

**Global Feature Representation:** This part combines all the packets of a flow to generate a global feature representation. First, we unfold  $X_L$  into  $N$  non-overlapping small blocks, known as patches  $p \in \{1, 2, \dots, P\}$ . Patches at the same position across different packets represent the same field within those packets. Extracting features from patches at the same position enables capturing the state transition information of the same field across packets, resulting in  $X_U \in \mathbb{R}^{P \times N \times D}$ . Here  $P = w \times h$  is the patch size,  $w \leq n$  and  $h \leq n$  are the height and width of the patch, respectively,  $N = \frac{HW}{P}$  is the number of patches, and the number of channels  $D$  is unchanged. Next, we

use the Transformer [24], [25] to compute global dependencies at corresponding positions between patches, generating  $X_T \in \mathbb{R}^{P \times N \times D}$ . We then fold  $X_T$  back into its original tensor arrangement to obtain  $X_F \in \mathbb{R}^{H \times W \times D}$ .

$$X_T(p) = \text{Transformer}(X_U(p)), 1 \leq p \leq P \quad (1)$$

In the experimental setup, we use the initial first  $l$  packets of each flow. To capture the global feature representation of the entire flow, we design a specific method: every  $r$  packets are arranged in a non-overlapping, cross-shaped configuration and combined into a tensor that serves as a channel. As an example, as illustrated in Fig. 5(b), suppose we process the initial first 12 packets of a bi-directional flow and splice every four packets together to form a tensor that serves as a channel. This approach results in a total of three channels. Finally, a  $p \times p$  patch is delineated on each channel (with  $p$  commonly set to 2 in the experiments), and Transformer operation is used to perform global feature extraction at the corresponding position of each patch.

**Feature Fusion:** In this process,  $X_F$  is projected into a low  $C$ -dimensional space using a point-wise convolution and then combined with  $X$  via a concatenation operation. These concatenated features are subsequently fused by another CNN layer, producing  $X' \in \mathbb{R}^{H \times W \times C}$ .

### C. Single View Opinion Generator

In order to quantify the degree of reliability of the classification results output by the classifier, we express the reliability of these results in a convenient and intuitive probabilistic manner. We design the opinion generator specifically to add a reliability metric to the classification results of the classifier. In this context, the opinion refers to the outcome produced by the opinion generator, which includes both the classification decision and uncertainty value that indicates the reliability of that decision. In this module, we use uncertainty estimation to measure the reliability of the classification decision for each view. This is achieved by modeling second-order classification probabilities, which form a distribution of classification probability distributions with uncertainty. Since the softmax function inflates the probability values, it can easily output high probability values for the unknown classes in open-set scenarios [12], [13], leading to the misclassification of unknown classes as known ones. To avoid the above problem, we replace

the softmax function with a softplus layer and introduce the Dirichlet distribution to uncertainty estimation, which helps in constructing the distribution of the classification probability distribution [26], [27].

More specifically, for the  $v$ -th view, assume there are  $K$  classes. The value derived from the softplus layer serves as a measure of support for classifying the sample into a specific class  $k$  (i.e., Evidence  $e^v = [1, \dots, e_k^v, \dots, e_K^v]$ ), where  $e_k^v \geq 0$  represents the value of evidence learned by the model to classify this sample into the  $k$ -th class. Parameterizing the Dirichlet distribution  $\alpha^v = [1, \dots, \alpha_k^v, \dots, \alpha_K^v]$  (where  $\alpha_k^v = e_k^v + 1$ ) with evidence, the Dirichlet distribution is given by:

$$D(\mathbf{p} | \alpha) = \begin{cases} \frac{1}{B(\alpha)} \prod_{k=1}^K p_k^{\alpha_k-1} & \text{for } \mathbf{p} \in \mathcal{S}_K \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The Dirichlet distribution is a probability density function for possible values of the probability mass function (PMF)  $\mathbf{p}$ . Here,  $B(\alpha)$  is the  $K$ -dimensional beta function, and  $\mathcal{S}_K$  is the Dirichlet strength, and  $\mathcal{S}_K = \{\mathbf{p} | \sum_{k=1}^K p_k = 1 \text{ and } 0 \leq p_1, \dots, p_K \leq 1\}$ .

We use the Dirichlet distribution to quantify the probability density of classification, treating each classification decision from different views as distinct opinions. This method assigns a belief score  $b$  to each class and introduces uncertainty  $u$  to the classification decision.

$$u^v + \sum_{k=1}^K b_k^v = 1 \quad (3)$$

where  $u^v \geq 0$  and  $b_k^v \geq 0$  ( $k = 1, \dots, K$ ) represent decision result uncertainty and the belief score (i.e. probability) of class  $k$ -th, respectively. The calculation of these values is expressed as follows:

$$b_k^v = \frac{e_k^v}{S^v}, u^v = \frac{K}{S^v} \quad (4)$$

$$S^v = \sum_{i=1}^K e_i^v + 1 = \sum_{i=1}^K \alpha_i^v$$

where  $S^v$  is the Dirichlet intensity at the  $v$ -th view.

When the classifier detects increased evidence for the  $k$ -th class in a sample, both the probability and belief score for that class increase, thereby reducing the uncertainty of its classification decision. Conversely, when evidence is scant, the belief score approaches zero, which increases the overall uncertainty of the classification decision, tending it toward one on the scale. Ideally, for samples from known classes, the uncertainty of their decisions is close to zero due to prior knowledge, while for samples from unknown classes, the uncertainty is significantly higher than zero due to having never been seen. For class  $k$  in the current view  $v$ , the expected probability  $\hat{p}_k^v$  of an opinion is equal to the projection of the Dirichlet distribution onto the base, i.e.,  $\hat{p}_k^v = \frac{\alpha_k^v}{S^v}$  [28].

#### D. Multi-View Opinion Fusion

Since the reliability of classification decisions varies across different views and samples, we need to assess the reliability of

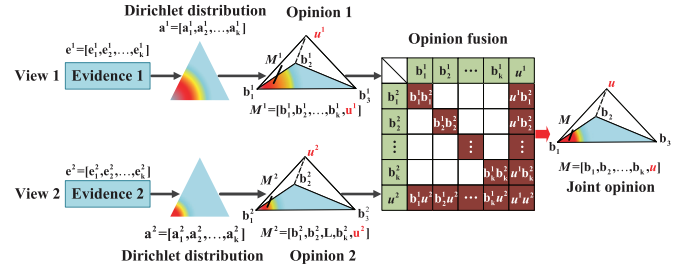


Fig. 6. Fuse the classification opinions of the two views and use the three vertices of the triangle to represent the three classes. Within the triangle, red indicates high probability density, while blue signifies low probability density.

decisions for each view in each sample adaptively. Therefore, it is necessary to dynamically fuse the classification decisions from all views based on the uncertainty in the opinions. In subsection III-A, we partition a flow into three views (i.e., IP Header, Transport Layer Header, and Packet Payload) to learn the opinions  $\mathcal{M}^v = \{\{b_k^v\}_{k=1}^K, u^v\}$  for each view separately, each consisting of the classification decision and its associated uncertainty, which serves as an indicator of the decision's reliability. In order to fuse the three views with classification decision uncertainty to form a reliable final joint opinion  $\mathcal{M} = \{\{b_k\}_{k=1}^K, u\}$ , we introduce the Dempster-Shafer evidence theory [26], [29], [30] to fuse the classification opinions from different sources.

As illustrated in Fig. 6, the figure presents an example of fusing opinions from two views,  $\mathcal{M}^1 = \{\{b_k^1\}_{k=1}^K, u^1\}$  and  $\mathcal{M}^2 = \{\{b_k^2\}_{k=1}^K, u^2\}$ . Each view gathers evidence  $e$  and parameterizes the Dirichlet distribution to obtain the view with uncertainty  $u$  and belief score  $b$  of classification decision. Ultimately, the final joint opinion  $\mathcal{M}$  is then derived using the fusion criterion.

$$\mathcal{M} = \mathcal{M}^1 \oplus \mathcal{M}^2 \quad (5)$$

The specific belief score and uncertainty are given by:

$$b_k = \frac{1}{1-C} (b_k^1 b_k^2 + b_k^1 u^2 + b_k^2 u^1), u = \frac{1}{1-C} u^1 u^2 \quad (6)$$

$C = \sum_{i \neq j} b_i^1 b_j^2$  represents a measure of the degree of conflict between two probability score sets.  $\frac{1}{1-C}$  is the normalization factor.

The joint opinion  $\mathcal{M}$  from the three views is also derived based on the aforementioned fusion criterion. The subsequent evidence and Dirichlet distribution parameters are induced as:

$$S = \frac{K}{u}, e_k = b_k \times S, \alpha_k = e_k + 1 \quad (7)$$

To more concretely illustrate the fusion of opinions from multiple views, we present the example of two views in open-set scenarios. When there is a lack of evidence favoring classification in both views, this leads to low joint belief scores and final prediction probabilities, and increased joint uncertainty in the classification opinions. In contrast, if both views receive sufficient evidence, the outcome is high joint belief scores and reliable, high-scoring final prediction probabilities, alongside reduced joint uncertainty. When a significant disparity exists in the amount of classification evidence between the

two views, we give preference to the view with the stronger evidence. Thus, the joint uncertainty for unknown samples, which the classifier has never encountered, is significantly higher than for known classes. Based on the aforementioned analysis, the joint uncertainty between known and unknown samples varies greatly—specifically, the classification decision uncertainty for known samples is considerably lower than for unknown samples. Consequently, we apply this principle to classify known and unknown classes in the open-set scenario.

#### E. Joint Loss

In this subsection, we construct our loss function. The common cross-entropy loss function is:

$$\mathcal{L}_{ce} = - \sum_{j=1}^K y_{ij} \log(p_{ij}), \quad (8)$$

where  $p_{ij}$  is the prediction probability that the  $i$ -th sample is predicted to be class  $j$ . Since we use the Dirichlet distribution to simulate the distribution of the classification probability distribution, and we denote the loss function after amending Eq. 8 as:

$$\begin{aligned} \mathcal{L}_{ace}(\alpha_i) &= \int \left[ \sum_{j=1}^K -y_{ij} \log(p_{ij}) \right] \frac{1}{B(\alpha_i)} \prod_{j=1}^K p_{ij}^{\alpha_{ij}-1} d\mathbf{p}_i \\ &= \sum_{j=1}^K y_{ij} (\psi(S_i) - \psi(\alpha_{ij})) \end{aligned} \quad (9)$$

where  $\psi(\cdot)$  is the digamma function and  $\mathcal{L}_{ace}$  represents the integral of the cross-entropy on the Dirichlet distribution.

To differentiate the classification evidence generated by data samples according to their contributions to the classification, more evidence is produced for the correct labels of the known classes, while less evidence is generated for the open-set unknown classes. We penalize unknown classes by using KL divergence to produce less evidence of classification. The KL divergence term is denoted as:

$$\begin{aligned} &KL[D(\mathbf{p}_i | \tilde{\alpha}_i) \| D(\mathbf{p}_i | \mathbf{1})] \\ &= \log \left( \frac{\Gamma(\sum_{k=1}^K \tilde{\alpha}_{ik})}{\Gamma(K) \prod_{k=1}^K \Gamma(\tilde{\alpha}_{ik})} \right) \\ &\quad + \sum_{k=1}^K (\tilde{\alpha}_{ik} - 1) \left[ \psi(\tilde{\alpha}_{ik}) - \psi\left(\sum_{j=1}^K \tilde{\alpha}_{ij}\right) \right] \end{aligned} \quad (10)$$

where  $\Gamma(\cdot)$  is the gamma function and  $\tilde{\alpha}_i = y_i + (1 - y_i) \odot a_i$  is the adjustment parameter of the Dirichlet distribution that is set to prevent the evidence of true label from being penalized to zero. Consequently, the loss function of a single view flow feature is then:

$$\mathcal{L}(\alpha_i) = \mathcal{L}_{ace}(\alpha_i) + \lambda_t KL[D(\mathbf{p}_i | \tilde{\alpha}_i) \| D(\mathbf{p}_i | \mathbf{1})] \quad (11)$$

where  $\lambda_t > 0$  is the balance factor. Combining the classification opinions of flow characteristics from three views, the final joint classification loss function is:

$$\mathcal{L}_{all} = \sum_{i=1}^N \left[ \mathcal{L}(\alpha_i) + \sum_{v=1}^3 \mathcal{L}(\alpha_i^v) \right] \quad (12)$$

#### IV. CLASSIFICATION PHASE OF RoNeTC

The aim of the classification phase is to identify unknown classes and accurately classify known classes into their respective corresponding label sets. After the training phase is complete, the feature extractors for the three views and the opinion fusion module, which includes the joint classification decision and its associated uncertainty, are utilized. This uncertainty refers to the reliability of the decision. Subsequently, the knowledge gained from the aforementioned processes is used in the open-set classification phase. In this phase, the threshold is determined using joint uncertainty to distinguish between known and unknown classes. Uncertainty hinges on the characteristic that flows with higher belief scores in classification and more reliable classification outcomes have correspondingly lower uncertainty, thus being classified into the known class. Conversely, flows with lower belief scores and unreliable classification results exhibit correspondingly higher uncertainty, hence being classified into the unknown class.

To facilitate the identification of the optimal threshold, we draw inspiration from Youden's index [31]. We define a function  $\eta(\sigma) = 2 * TPRs(\sigma) - FPRs(\sigma)$  based on the true positive rates (TPRs) and false positive rates (FPRs) of the samples. The optimal uncertainty threshold is then determined by identifying the threshold  $\hat{\sigma}$  that maximizes the function, i.e.,  $\hat{\sigma} = \arg\max_{\sigma} \eta(\sigma)$ , and considering the range around this threshold.

#### V. EXPERIMENTAL EVALUATION

In this section, we detail the experimental setup, parameter selection and ablation study of RoNeTC.

##### A. Experimental Settings

1) *Dataset*: We use three publicly available datasets for comprehensive evaluation in both closed-set and open-set settings. Detailed information on each dataset is provided in Table II.

**Dataset-I**: The *UNSW traffic dataset* was sourced from different web-based devices and collected and stored in a smart device environment built on campus by Sivanathan et al. [32]. The data collection period spanned from October 1, 2016, to April 13, 2017, covering approximately 26 weeks, and was organized into different pcaps by date. The selection criteria for the classes specified exactly 1,000 samples per class.

**Dataset-II**: The *CIC dataset (2022)* [33] is utilized for the behavioral analysis and vulnerability testing of various IoT devices that operate on protocols such as IEEE 802.11, Zigbee, and Z-Wave. We select 500 samples for each class.

**Dataset-III**: The *Mapps dataset* [34], derived from the 'MApps' public mobile application traffic dataset, was collected by Pham et al. in 2021. The authors recruited ten student volunteers to collect traffic consistently over six months using eight different smartphones. We chose flow numbers ranging from 500 to 1,000 per class.

To rigorously and comprehensively evaluate the reliability and robustness of RoNeTC in open-set settings, we elaborately established six unknown scenarios reflecting open-set

TABLE II  
BRIEF DESCRIPTION OF THE DATASET

| (a) <i>Dataset-I</i> |                 |    |                   |    |                   |
|----------------------|-----------------|----|-------------------|----|-------------------|
| ID                   | label           | ID | label             | ID | label             |
| 0                    | TP-Link *       | 1  | Withings * sensor | 2  | Netatmo * station |
| 3                    | Belkin * sensor | 4  | Insteon Camera    | 5  | Netatmo Welcome   |
| 6                    | Belkin * switch | 7  | Withings *        | 8  | Amazon Echo       |
| 9                    | Samsung *       | 10 | PIX-STAR *        | 11 | Samsung * Tab     |

| (b) <i>Dataset-II</i> |                 |    |                 |    |                 |
|-----------------------|-----------------|----|-----------------|----|-----------------|
| ID                    | label           | ID | label           | ID | label           |
| 0                     | HeimVision *    | 1  | Google Nest *   | 2  | Philips *       |
| 3                     | AMCREST *       | 4  | Ring Base *     | 5  | Amazon Plug     |
| 6                     | Arlo * Camera   | 7  | Atomi Coffee *  | 8  | Arlo * Station  |
| 9                     | Borun/Sichuan * | 10 | SIMCAM 1S *     | 11 | Smart Board     |
| 12                    | Home * Camera   | 13 | Netatmo Camera  | 14 | Amazon * Spot   |
| 15                    | Amazon * Dot    | 16 | Sonos * Speaker | 17 | Amazon * Studio |

| (c) <i>Dataset-III</i> |             |    |                 |    |              |
|------------------------|-------------|----|-----------------|----|--------------|
| ID                     | label       | ID | label           | ID | label        |
| 0                      | vnexpress   | 1  | reddit          | 2  | toc_chien    |
| 3                      | nhaccuatui  | 4  | zingmp3         | 5  | phim247      |
| 6                      | spotify     | 7  | voizfm          | 8  | olaparty     |
| 9                      | wikipedia   | 10 | starmarker      | 11 | baohay24h    |
| 12                     | jobway      | 13 | soundcloud      | 14 | partying     |
| 15                     | truyenaudio | 16 | messenger       | 17 | tango        |
| 18                     | iQIYI       | 19 | tuotire_online  | 20 | vietnamworks |
| 21                     | dan_tri     | 22 | shoppe          | 23 | weeboo       |
| 24                     | nonolive    | 25 | lienquan_mobile | 26 | tinmoi24h    |
| 27                     | kaka        | 28 | medoctruyen     | 29 | quora        |
| 30                     | tinhte      | 31 | lotus           | 32 | chotot       |
| 33                     | comico      | 34 | lazada          | 35 | bigo         |
| 36                     | topcv       | 37 | thanhvien       | 38 | soha         |
| 39                     | wetv        | 40 | podcast_player  | 41 | radiofm      |
| 42                     | zalo        | 43 | freefire        | 44 | tiki         |
| 45                     | likee_lite  | 46 | sendo           |    |              |

The symbol '\*' indicates abbreviation.

TABLE III  
OPEN-SET SCENARIOS DIVISION

| Scenario   | Known Classes      | Unknown Classes    |
|------------|--------------------|--------------------|
| Scenario-A | <i>Dataset-I</i>   | <i>Dataset-II</i>  |
| Scenario-B | <i>Dataset-I</i>   | <i>Dataset-III</i> |
| Scenario-C | <i>Dataset-II</i>  | <i>Dataset-I</i>   |
| Scenario-D | <i>Dataset-II</i>  | <i>Dataset-III</i> |
| Scenario-E | <i>Dataset-III</i> | <i>Dataset-II</i>  |
| Scenario-F | <i>Dataset-III</i> | <i>Dataset-I</i>   |

conditions based on three datasets, as detailed in Table III. For clarity, we take Scenarios-A as an example. We use the entire set of *Dataset-I* as known samples, partitioning them into training, validation, and test sets that represent 60%, 20%, and 20% of the number of each class, respectively. In the context of open-set testing, all samples from *Dataset-II* in Scenario-A are treated as unknown classes and included in the test sets.

2) *Metrics*: We define the metrics employed to evaluate RoNetC, specifically *Recall*, *Precision*, and *F-macro*, which will hereafter be referred to as *Rec*, *Pre*, and *F1*, respectively. The source code is available at: <https://github.com/xuemanxm/RoNetC/tree/main>.

### B. Parameter Selection

In order to achieve better classification performance, we optimize the experimental setup through parameter selection experiments. We capture the initial first  $l$  packets of each bi-directional flow, where  $l \in \{4, 8, 12, 16\}$ , and intercept the initial first  $b$  bytes of each packet, where  $b \in \{64, 128, 256\}$ . According to Section III-A, we partition the packet into three components based on the TCP/IP protocol

stack: the IP Header, the Transport layer Header, and the Packet Payload—collectively referred to as the three views. In this way, we can perform a multi-view adaptive fusion of classification opinions, based on the classification decision and its uncertainty for each view of each flow, with the uncertainty indicating the reliability of that decision. This process results in the final classification decision and its associated uncertainty, which are then applied to the classification tasks in open-set scenarios. We select appropriate parameters for open-set scenarios in six open-set settings (Scenario-A to Scenario-F), with the experimental results displayed in Table IV. Its experimental results in the conventional closed-set environment are shown in Table V.

1) *Open-Set Network Traffic Classification*: In the following, we evaluate the effects of the parameter experiments for each of the six open-set scenarios, and the results are shown in Table IV. As the number of bytes  $b$  and the number of packets  $l$  increase, the *F1* also increases. However, in Scenario-A, Scenario-C, and Scenario-F, where  $b$  is 128 and 256, the increase in the size of the input tensor leads to the model capturing more useless features along with useful ones. Furthermore, as the number of packets  $l$  increases, the dimension of the input tensor also increases, leading to a rise in the volume of the input space. This results in sparser data and higher computational complexity. Therefore, in all these scenarios, as the values of  $b$  and  $l$  increase, the *F1* initially increase and then decrease.

In Scenario-A, we design *Dataset-I* as the known class sample set and *Dataset-II* as the unknown class sample set. We select 128 bytes ( $b = 128$ ) and 8 packets ( $l = 8$ ) as the optimal parameters for the current scenario. As demonstrated in Table IV, this configuration results in the highest open-set classification *F1* of 96.44% among various combinations of byte and packet counts. Specifically, this parameter set achieves a *Rec* of 95.92% and a *Pre* of 97.22%. Scenario-B, in alignment with the known class samples of Scenario-A, employs *Dataset-I*, whereas the unknown class samples are derived from *Dataset-III*. For this configuration, we choose 256 bytes ( $b = 256$ ) and 16 packets ( $l = 16$ ) as the optimal parameters. With these settings, the classification *F1* reaches a maximum value of 98.16%, and its corresponding *Rec* = 98.11% and *Pre* = 98.32% are also close to the maximum value. Although Scenario-B and Scenario-A share the same known classes, the open-set classification performance of Scenario-B surpasses that of Scenario-A. This improvement is attributed to the greater disparity between the known and unknown class samples in Scenario-B, making them easier to distinguish.

The known class samples for both Scenarios-C and Scenarios-D are sourced from *Dataset-II*. In Scenario-C, the unknown class samples are taken from *Dataset-I* and are entirely categorized as the test set. For this scenario, we select 256 bytes ( $b = 256$ ) and 4 packets ( $l = 4$ ) as the optimal parameters. Under these conditions, the open-set classification achieves a maximum *F1* of 94.56%. Scenario-D is based on *Dataset-III* as the unknown class, and  $b = 256$ ,  $l = 16$  is selected as the best parameter for this scenario, based on the criterion of maximum classification *F1*, which is 94.03%,



TABLE IV  
EXPERIMENTAL RESULTS OF OPEN-SET PARAMETER SELECTION

|            | $b$     | $l$ | 64    |       |       |       | 128   |              |       |       | 256          |       |       |              |
|------------|---------|-----|-------|-------|-------|-------|-------|--------------|-------|-------|--------------|-------|-------|--------------|
|            |         |     | 4     | 8     | 12    | 16    | 4     | 8            | 12    | 16    | 4            | 8     | 12    | 16           |
| Scenario-A | Rec (%) |     | 97.50 | 97.17 | 96.02 | 95.55 | 95.97 | 95.92        | 95.97 | 94.21 | 95.60        | 94.40 | 94.27 | 98.03        |
|            | Pre (%) |     | 93.00 | 93.23 | 92.22 | 93.02 | 94.24 | 97.22        | 94.41 | 94.84 | 96.53        | 96.27 | 94.44 | 94.14        |
|            | F1 (%)  |     | 93.33 | 92.95 | 91.57 | 92.04 | 94.09 | <b>96.44</b> | 94.03 | 94.11 | 95.96        | 95.29 | 92.65 | 94.57        |
| Scenario-B | Rec (%) |     | 97.03 | 97.20 | 85.07 | 95.06 | 96.32 | 95.99        | 94.32 | 92.47 | 95.33        | 96.80 | 96.14 | 98.11        |
|            | Pre (%) |     | 89.32 | 87.78 | 93.83 | 91.66 | 96.25 | 98.70        | 94.56 | 94.99 | 97.11        | 98.72 | 94.48 | 98.32        |
|            | F1 (%)  |     | 90.47 | 88.26 | 84.76 | 90.27 | 96.06 | 97.30        | 93.39 | 93.55 | 96.16        | 97.68 | 94.24 | <b>98.16</b> |
| Scenario-C | Rec (%) |     | 88.96 | 87.58 | 88.48 | 85.82 | 88.30 | 90.94        | 89.71 | 91.50 | 93.23        | 89.21 | 87.66 | 91.67        |
|            | Pre (%) |     | 95.93 | 95.61 | 91.11 | 97.42 | 95.14 | 98.53        | 97.09 | 94.94 | 96.80        | 99.37 | 92.76 | 98.50        |
|            | F1 (%)  |     | 91.56 | 90.70 | 87.02 | 90.27 | 90.13 | 94.35        | 92.84 | 92.42 | <b>94.56</b> | 92.92 | 89.71 | 94.48        |
| Scenario-D | Rec (%) |     | 88.78 | 85.20 | 84.56 | 86.83 | 91.15 | 88.03        | 87.09 | 86.88 | 92.20        | 91.08 | 85.21 | 91.68        |
|            | Pre (%) |     | 96.56 | 95.84 | 94.37 | 95.30 | 96.53 | 94.12        | 94.14 | 95.26 | 95.91        | 93.60 | 94.43 | 97.27        |
|            | F1 (%)  |     | 92.53 | 89.16 | 87.65 | 90.31 | 93.44 | 90.19        | 89.72 | 90.02 | 93.40        | 90.40 | 88.95 | <b>94.03</b> |
| Scenario-E | Rec (%) |     | 86.18 | 81.42 | 81.77 | 81.46 | 86.14 | 81.59        | 83.70 | 83.14 | 90.69        | 87.65 | 89.46 | 90.77        |
|            | Pre (%) |     | 93.58 | 92.12 | 92.77 | 92.52 | 96.23 | 94.08        | 93.16 | 92.94 | 94.87        | 95.22 | 93.30 | 94.98        |
|            | F1 (%)  |     | 88.61 | 85.08 | 85.61 | 85.21 | 90.14 | 86.14        | 87.04 | 86.23 | 91.65        | 90.40 | 90.59 | <b>91.71</b> |
| Scenario-F | Rec (%) |     | 86.10 | 81.39 | 81.71 | 80.19 | 86.81 | 83.05        | 82.27 | 83.95 | 91.20        | 88.32 | 89.61 | 88.44        |
|            | Pre (%) |     | 92.32 | 92.54 | 91.94 | 92.68 | 96.31 | 92.09        | 95.19 | 94.23 | 96.58        | 97.22 | 97.26 | 96.52        |
|            | F1 (%)  |     | 87.28 | 84.93 | 84.67 | 84.35 | 90.53 | 85.83        | 87.32 | 87.40 | <b>93.26</b> | 92.14 | 92.90 | 91.36        |

TABLE V  
EXPERIMENTAL RESULTS OF CLOSED-SET PARAMETER SELECTION

| $b$ | $l$ | Dataset-I |         |              | Dataset-II |         |              | Dataset-III |         |              |
|-----|-----|-----------|---------|--------------|------------|---------|--------------|-------------|---------|--------------|
|     |     | Rec (%)   | Pre (%) | F1 (%)       | Rec (%)    | Pre (%) | F1 (%)       | Rec (%)     | Pre (%) | F1 (%)       |
| 64  | 4   | 99.88     | 99.88   | 99.88        | 98.28      | 98.30   | 98.27        | 93.28       | 94.69   | 93.68        |
|     | 8   | 99.71     | 99.71   | 99.71        | 98.06      | 98.09   | 98.06        | 90.68       | 92.30   | 91.13        |
|     | 12  | 99.83     | 99.83   | 99.83        | 97.44      | 97.50   | 97.46        | 90.45       | 91.80   | 90.73        |
|     | 16  | 99.50     | 99.50   | 99.50        | 96.89      | 96.92   | 96.90        | 91.00       | 92.78   | 91.51        |
| 128 | 4   | 99.83     | 99.83   | 99.83        | 99.11      | 99.11   | 99.11        | 92.37       | 94.17   | 92.92        |
|     | 8   | 99.96     | 99.96   | <b>99.96</b> | 98.11      | 98.14   | 98.11        | 92.97       | 94.90   | 93.52        |
|     | 12  | 99.75     | 99.75   | 99.75        | 97.72      | 97.86   | 97.74        | 91.48       | 93.67   | 92.12        |
|     | 16  | 99.62     | 99.63   | 99.63        | 97.22      | 97.25   | 97.21        | 92.28       | 94.59   | 93.01        |
| 256 | 4   | 99.92     | 99.92   | 99.92        | 99.11      | 99.13   | <b>99.11</b> | 94.02       | 95.82   | 94.56        |
|     | 8   | 99.96     | 99.96   | 99.96        | 98.67      | 98.68   | 98.66        | 93.74       | 95.78   | 94.21        |
|     | 12  | 99.79     | 99.80   | 99.79        | 98.50      | 98.52   | 98.50        | 94.39       | 96.04   | <b>94.88</b> |
|     | 16  | 99.83     | 99.84   | 99.83        | 98.83      | 98.85   | 98.84        | 93.54       | 96.02   | 94.19        |

corresponding to *Rec* of 91.68%, *Pre* of 97.27%. Scenarios-C and D share the same known classes; however, Scenario-C has approximately one-third the number of unknown samples compared to Scenario-D. Consequently, the *F1* for Scenario-C is slightly higher than that of Scenario-D, despite the identical known classes.

Scenarios-E and F use *Dataset-III* as a known sample set. In Scenario-E, *Dataset-II* is the unknown sample set, achieving the highest classification *F1* of 91.71%, which corresponds to optimal parameters  $b = 256$  and  $l = 16$ . However, in Scenario-F, *Dataset-I* serves as the unknown class. Using an *F1* of 93.26% as the benchmark, the optimal parameters selected are  $b = 256$  and  $l = 4$ . Given that Scenarios-E and F have a significantly larger number of known classes compared to other scenarios, they exhibit the lowest classification *F1*. However, the larger quantity of unknown class samples in Scenario-E than in Scenario-F introduces additional classification interference, resulting in the lowest effect among the six open-set classifications observed in Scenario-E. In contrast, Scenario-B, with its smaller number of known classes and greater disparity between known and unknown class samples, achieves the highest classification *F1* of 98.16% in the open-set classification.

**Uncertainty Analysis:** As depicted in Fig. 7, we present a plot illustrating the uncertainty distribution for both known and unknown samples within the open-set context. In subsection III-D and section IV, we delve into the optimal

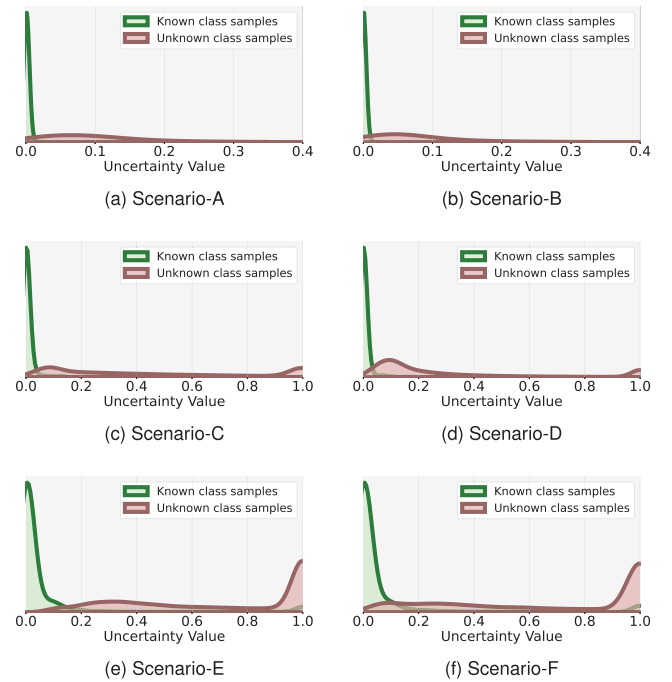


Fig. 7. Uncertainty distribution KDE figures for Known and Unknown Classes in six open-set scenarios.

characteristics for both known and unknown classes. Ideally, known classes should exhibit high belief scores and low uncertainty, while the optimal scenario for unknown classes is characterized by low belief scores and high uncertainty. The uncertainty values of all known samples in the graph are tightly clustered around the scale 0. However, in Fig. 7(c)(f), there is a slightly looser distribution of known classes due to the large number of known classes and the best effect of known class classification is only 94.88%. Nevertheless, these values remain tightly concentrated on the left side of the x-axis. In contrast, the uncertainty values of the unknown class samples should be far from the 0 point of the scale due to their lower belief scores. This is indeed the case for the unknown classes

TABLE VI  
RESULTS OF VIEWS ABLATION EXPERIMENTS

| ID | Style    | IP Header | Transport Layer Header | Packet Payload | <i>FI</i> (%) |               |               |               |               |               |
|----|----------|-----------|------------------------|----------------|---------------|---------------|---------------|---------------|---------------|---------------|
|    |          |           |                        |                | Scenario-A    | Scenario-B    | Scenario-C    | Scenario-D    | Scenario-E    | Scenario-F    |
| 1  | RoNeTC-S | ✓         |                        |                | 89.06(↓ 7.38) | 75.97(↓22.19) | 75.54(↓19.02) | 68.83(↓25.20) | 79.07(↓12.64) | 75.98(↓17.28) |
| 2  |          |           | ✓                      |                | 85.50(↓10.94) | 86.00(↓12.16) | 70.33(↓24.23) | 69.15(↓24.88) | 74.93(↓16.78) | 77.69(↓15.58) |
| 3  |          |           |                        | ✓              | 90.37(↓ 6.07) | 87.12(↓11.04) | 81.20(↓13.36) | 75.27(↓18.77) | 91.68(↓ 0.03) | 93.04(↓ 0.22) |
| 4  |          | +         | +                      | +              | 93.93(↓ 2.51) | 92.82(↓ 5.34) | 91.40(↓ 3.16) | 84.03(↓10.00) | 89.24(↓ 2.47) | 92.56(↓ 0.70) |
| 5  | RoNeTC-D | ✓         |                        | ✓              | 93.09(↓ 3.35) | 92.08(↓ 6.08) | 94.35(↓ 0.21) | 92.83(↓ 1.20) | 85.94(↓ 5.77) | 90.61(↓ 2.65) |
| 6  |          |           | ✓                      | ✓              | 94.15(↓ 2.29) | 92.01(↓ 6.15) | 86.95(↓ 7.61) | 85.16(↓ 8.87) | 82.84(↓ 8.87) | 84.84(↓ 8.42) |
| 7  | RoNeTC   | ✓         | ✓                      |                | 90.46(↓ 5.98) | 84.56(↓13.60) | 87.47(↓ 7.09) | 78.72(↓15.31) | 85.87(↓ 5.84) | 93.13(↓ 0.13) |
| 8  |          | ✓         | ✓                      | ✓              | <b>96.44</b>  | <b>98.16</b>  | <b>94.56</b>  | <b>94.03</b>  | <b>91.71</b>  | <b>93.26</b>  |

Multi-view ablation experiment results. The symbol ✓ indicates that the corresponding data was used to construct the corresponding view for experimentation. It's important to note that the '+' symbol in 'RoNeTC-S' signifies that the three sets of data (IP Header, Transport Layer Header, and Packet Payload) are combined to form a complete single entity and tested as one view.

TABLE VII  
RESULTS OF CHANNEL ABLATION EXPERIMENTS

| Method     | RoNeTC-O <i>FI</i> (%) | RoNeTC <i>FI</i> (%) |
|------------|------------------------|----------------------|
| Scenario-A | 95.24(↓ 1.20)          | <b>96.44</b>         |
| Scenario-B | 96.54(↓ 1.62)          | <b>98.16</b>         |
| Scenario-C | 93.41(↓ 1.15)          | <b>94.56</b>         |
| Scenario-D | 89.10(↓ 4.93)          | <b>94.03</b>         |
| Scenario-E | 87.72(↓ 3.99)          | <b>91.71</b>         |
| Scenario-F | 84.82(↓ 8.44)          | <b>93.26</b>         |

in the subfigure, all of which are essentially distant from the 0 scale. This presents a stark contrast with the known classes, which are densely clustered around the 0 scale. Especially in Fig. 7(c)(f), the uncertainty values of certain unknown classes cluster around scale 1, providing a clear distinction from the known classes.

2) *Closed-Set Network Traffic Classification*: Similarly, we conduct a parametric analysis in the conventional closed-set environment, defined as the closed-set environment containing only known classes. The experimental results for the three closed-set scenarios are presented in Table V. On *Dataset-I*, all combinations of the number of bytes and packets achieve excellent classification results, with *FI*, *Rec*, and *Pre* all above 99%. The optimal results are observed in closed-set network traffic classification using parameter settings of 128 bytes ( $b = 128$ ) and 8 packets ( $l = 8$ ), where all three metrics reach 99.96%. On *Dataset-II*, the effectiveness of traffic classification gradually improves as the number of bytes increases. The optimal performance is achieved with a byte count of 256 ( $b = 256$ ) and a packet length of 4 ( $l = 4$ ), recording 99.11% for both *FI* and *Rec*, and 99.13% for *Pre*.

As the number of classes increases and classification complexity rises, there is a noticeable decrease in the overall network traffic classification results. On *Dataset-III*, which has the highest number of classes, the classification results are slightly weaker compared to the previous two closed-set datasets. However, with parameters set at  $b = 256$  and  $l = 12$ , it still achieves an *FI* of 94.88%, a *Rec* of 94.39%, and a *Pre* of 96.04%.

### C. Ablation Study

To evaluate the effectiveness of the RoNeTC dynamic multi-view uncertainty assessment and the global packet splicing approach within the global-local Transformer module, we

conduct extensive ablation studies on these two features. We assess the experimental outcomes using the *FI*, and the results are presented in Tables VI and Tables VII.

1) *Multi-View Ablation Studies*: We analyze the ablation effects on the IP Header, Transport Layer Header, and Packet Payload by considering single-view and dual-view permutations, denoted as RoNeTC-S and RoNeTC-D, respectively. The results are presented in Table VI. Notably, among the three broad classes—RoNeTC-S, RoNeTC-D, and RoNeTC—our proposed three-view scheme, RoNeTC, consistently achieves the highest *FI* in all open-set scenarios, demonstrating superior performance. On the contrary, the single-view RoNeTC-S exhibits the largest decline and the poorest performance in open-set classification. Although it splices three parts into a single view using the same amount of information as RoNeTC, the performance still suffers from varying degrees of degradation. The results clearly demonstrate that employing multiple views to finely handle uncertainty estimation for each view and enhance their reliability proves more effective for open-set classification.

2) *Packet Splicing Methods for Global Feature Representation*: In this part, each packet in the flow is treated as an individual tensor to form a channel, which we refer to as RoNeTC-O. The results are presented in Table VII. As described in subsection III-B, the global feature representation illustrated in Fig. 5(b) employs a splice approach that arranges every  $p$  packets of a flow in a non-overlapping, cross-shaped arrangement into a tensor and treats it as a channel.

Our experiments show that RoNeTC achieves the best performance in all six scenarios. More specifically, the *FI* of RoNeTC-O decline in all open-set scenarios relative to RoNeTC, indicating poorer performance in open-set classification. Most notably, the *FI* in Scenario-F decreases significantly, highlighting that RoNeTC-O struggles in complex situations with numerous known classes and ambiguous identification of unknown classes. Our findings affirm that RoNeTC, based on the principle that the correlation between packets is greater than within packets, delivers superior performance and greater robustness in challenging open-set environments.

## VI. EXPERIMENTAL COMPARISON WITH SOTA METHODS

In this section, in order to fully evaluate the reliability and robustness of our proposed method, RoNeTC, we

TABLE VIII  
COMPARISON WITH THE SOTA METHODS IN THE  
CLOSED-SET SCENARIOS

| Datasets           | Method | Rec (%)       | Pre (%)       | F1 (%)        |
|--------------------|--------|---------------|---------------|---------------|
| <i>Dataset-I</i>   | ETC-PS | 99.25(↓ 0.71) | 99.25(↓ 0.71) | 99.25(↓ 0.71) |
|                    | AutoUA | 99.53(↓ 0.43) | 99.56(↓ 0.40) | 99.54(↓ 0.42) |
|                    | GradBP | 99.71(↓ 0.25) | 99.71(↓ 0.25) | 99.71(↓ 0.25) |
|                    | RoNeTC | 99.96         | 99.96         | <b>99.96</b>  |
| <i>Dataset-II</i>  | ETC-PS | 96.11(↓ 3.00) | 96.50(↓ 2.62) | 96.08(↓ 3.03) |
|                    | AutoUA | 97.73(↓ 1.38) | 97.78(↓ 1.35) | 97.74(↓ 1.37) |
|                    | GradBP | 97.44(↓ 1.67) | 97.63(↓ 1.50) | 97.44(↓ 1.67) |
|                    | RoNeTC | 99.11         | 99.13         | <b>99.11</b>  |
| <i>Dataset-III</i> | ETC-PS | 90.34(↓ 4.05) | 90.67(↓ 5.37) | 90.45(↓ 4.43) |
|                    | AutoUA | 87.86(↓ 6.53) | 87.92(↓ 8.12) | 87.77(↓ 7.11) |
|                    | GradBP | 91.97(↓ 2.42) | 92.02(↓ 4.02) | 91.94(↓ 2.94) |
|                    | RoNeTC | 94.39         | 96.04         | <b>94.88</b>  |

conduct experimental comparisons with state-of-the-art (SOTA) open-set network traffic classification methods in two major scenarios: open-set and closed-set scenarios. The SOTA methods for network traffic classification in open-set scenarios are as follows:

- ETC-PS [5]: ETC-PS constructs traffic paths using sequences of packet lengths, applies path transformations to extract information, and finally computes multi-scale path signatures to handle both open-set and closed-set network traffic methods.
- AutoUA [4]: AutoUA is an autonomous learning framework for open-set network traffic classification. It uses the first  $l$  packets of each flow as input to build a CNN network to identify network traffic. In the original paper, AutoUA plots the Cumulative Distribution Function (CDF) based on probability values, using the probability values on the CDF corresponding to the  $1 - \epsilon$  ( $\epsilon$  represents *Accuracy*) of known classes as thresholds to distinguish between known and unknown classes.
- GradBP (max): GradBP [1] is a method for evaluating feed-forward inference using deep learning gradient back-propagation. It selects the gradient of the input layer from the first step of backpropagation as a threshold to classify flows into known and unknown classes. In addition, GradBP is designed with two gradient thresholding methods. The first one is GradBP (max), where the gradient threshold is computed as the maximum value of the gradient.
- GradBP (square root): The second gradient threshold method of GradBP is GradBP (square root). This method is the same as GradBP (max), except that its gradient threshold is computed as the square root of the sum of the squares of the gradient.

#### A. Comparison on Closed-Set Network Traffic Classification Methods

We begin by comparing the performance of all methods on regular closed-set network traffic classification to observe the experimental results, displayed in Table VIII. We conduct an experimental comparison between RoNeTC and other SOTA open-set network traffic classification methods across three closed-set datasets. On all datasets, our method RoNeTC achieves the highest values on all metrics, demonstrating

TABLE IX  
COMPARISON WITH THE SOTA METHODS IN THE OPEN-SET SCENARIOS

| Scenario   | Method               | Rec (%)       | Pre (%)       | F1 (%)        |
|------------|----------------------|---------------|---------------|---------------|
| Scenario-A | ETC-PS               | 96.39(↑ 0.47) | 91.92(↓ 5.30) | 93.03(↓ 3.41) |
|            | AutoUA               | 91.80(↓ 4.12) | 48.33(↓48.89) | 52.22(↓44.22) |
|            | GradBP (max)         | 92.08(↓ 3.84) | 84.20(↓13.02) | 85.19(↓11.25) |
|            | GradBP (square root) | 59.83(↓36.09) | 81.06(↓16.16) | 63.31(↓33.13) |
|            | RoNeTC               | 95.92         | 97.22         | <b>96.44</b>  |
| Scenario-B | ETC-PS               | 95.66(↓ 2.45) | 80.53(↓17.79) | 84.32(↓13.84) |
|            | AutoUA               | 92.00(↓ 6.11) | 37.96(↓60.36) | 39.31(↓58.85) |
|            | GradBP (max)         | 92.08(↓ 6.03) | 58.90(↓39.42) | 66.15(↓32.01) |
|            | GradBP (square root) | 59.83(↓38.28) | 61.34(↓36.98) | 54.28(↓43.88) |
|            | RoNeTC               | 98.11         | 98.32         | <b>98.16</b>  |
| Scenario-C | ETC-PS               | 90.01(↓ 3.22) | 89.37(↓ 7.43) | 86.87(↓ 7.69) |
|            | AutoUA               | 92.77(↓ 0.46) | 38.19(↓58.61) | 40.76(↓53.80) |
|            | GradBP (max)         | 86.78(↓ 6.45) | 85.09(↓11.71) | 83.83(↓10.73) |
|            | GradBP (square root) | 56.06(↓37.17) | 86.98(↓ 9.82) | 61.47(↓33.09) |
|            | RoNeTC               | 93.23         | 96.80         | <b>94.56</b>  |
| Scenario-D | ETC-PS               | 89.88(↓ 1.80) | 90.04(↓ 7.23) | 88.25(↓ 5.78) |
|            | AutoUA               | 92.71(↑ 1.03) | 34.07(↓63.20) | 36.11(↓57.92) |
|            | GradBP (max)         | 86.78(↓ 4.90) | 65.02(↓32.25) | 69.50(↓24.53) |
|            | GradBP (square root) | 56.06(↓35.62) | 71.66(↓25.61) | 52.83(↓41.20) |
|            | RoNeTC               | 91.68         | 97.27         | <b>94.03</b>  |
| Scenario-E | ETC-PS               | 59.16(↓31.61) | 99.01(↑ 4.03) | 70.79(↓20.92) |
|            | AutoUA               | 85.90(↓ 4.87) | 55.84(↓39.14) | 64.42(↓27.29) |
|            | GradBP (max)         | 77.25(↓13.52) | 91.46(↓ 3.52) | 82.92(↓ 8.79) |
|            | GradBP (square root) | 63.28(↓27.49) | 94.10(↓ 0.88) | 80.31(↓11.40) |
|            | RoNeTC               | 90.77         | 94.97         | <b>91.71</b>  |
| Scenario-F | ETC-PS               | 76.18(↓24.02) | 96.64(↑ 0.06) | 76.67(↓16.59) |
|            | AutoUA               | 87.11(↓ 4.09) | 59.80(↓36.78) | 65.85(↓27.41) |
|            | GradBP (max)         | 77.25(↓13.95) | 84.74(↓11.84) | 77.34(↓15.92) |
|            | GradBP (square root) | 63.28(↓27.92) | 94.10(↓ 2.48) | 74.42(↓18.84) |
|            | RoNeTC               | 91.20         | 96.58         | <b>93.26</b>  |

superior closed-set classification performance. On *Dataset-I*, RoNeTC attains an optimal 99.96% across all three metrics. However, as the number of classes increases, the classification effectiveness of all methods decreases. For instance, in *Dataset-III*, which has the highest number of classes, all methods encounter the worst classification performance ability, such as AutoUA obtaining an *F1* of 87.77%. Nevertheless, RoNeTC maintains strong recognition capabilities, reaching an optimal value of 94.88%. The confusion matrix of RoNeTC is presented in the Appendix. Overall, our RoNeTC method consistently achieves excellent results in network traffic classification on closed-set scenarios.

#### B. Comparison on Open-Set Network Traffic Classification Methods

For the experimental comparison of network traffic classification for closed-set scenarios, the RoNeTC method achieves excellent classification results. Next, we will move to more complex open-set scenarios for further comparison. Our experimental comparison results with four state-of-the-art open-set network traffic classification methods across six open-set scenarios are shown in Table IX.

Our proposed method, RoNeTC, consistently achieves the highest values across all metrics, demonstrating superior performance in open-set classification compared to the other methods. Particularly in Scenario-B, RoNeTC excels with an impressive *F1* of 98.16%, which reflects excellent classification of known classes and robust recognition of both known and unknown classes in the open-set scenarios. RoNeTC also demonstrates excellent classification performance in the relatively complex Scenario-E and Scenario-F, achieving *F1* of 91.71% and 93.26%, respectively. Combining the results of previous closed-set comparison experiments, our RoNeTC

method achieves excellent classification results on both closed-set and open-set scenarios.

ETC-PS [5] processes open-set network traffic by performing binary classification to distinguish between known and unknown classes. To adapt this approach for identifying unknown classes and further classifying known classes, we plot the Cumulative Distribution Function (CDF) of the output probability values of ETC-PS. We select the point corresponding to the last flat region of the CDF, where the probability value stops increasing, as a threshold to distinguish the known from the unknown class. In Table IX, the *F1* of ETC-PS in each open-set scenarios is second only to our method, RoNeTC. It performs best in Scenario-A, achieving an *F1* of 93.03%, which is 3.41% lower than that of RoNeTC. The classification performance of ETC-PS decreases significantly as the number of known classes increases, with the worst results in Scenario-E, which has the largest number of known and unknown classes. In this scenario, ETC-PS achieves an *F1* of only 70.79%, which is 20.92% lower compare to our RoNeTC.

AutoUA [4] is utilized in open-set network traffic comparison experiments, with parameter settings that are consistent with the optimal parameters for each scenario, as derived from Table IV. The results of these experiments are shown in Table IX. Among all open-set scenarios, AutoUA achieved its best performance in Scenario-F, recording the highest *F1* of 65.85%, with a *Rec* of 87.11% and a *Pre* of 59.80%. This indicates superior recognition of known classes over unknown ones. Conversely, AutoUA was least effective in Scenario-D, where it recorded an *F1* of 36.11%, marking a significant decrease of 57.92% compared to our method RoNeTC. Overall, in open-set scenarios, AutoUA's recall consistently exceeds its precision. This indicates that while AutoUA is able to identify most flows of the known classes (high recall), it also incorrectly classifies many unknown class flows as known class flows (low precision). This suggests that AutoUA's ability to recognize unknown classes needs improvement.

GradBP [1] uses the sequence of packet lengths for the first 100 packets from the original bi-directional flow as input for all training data in open-set scenarios, as specified in the original paper. The threshold calculation is divided into two methods: GradBP (max) and GradBP (square root).

GradBP (max) achieves the best performance in Scenario-A among all the open-set scenarios, with 85.19% for *F1*, 92.08% for *Rec*, and 79.20% for *Pre*. This is the highest in both *F1* and *Rec* among all scenarios. However, compared to our proposed method, RoNeTC, the *F1* of GradBP (max) is still lower by 11.25%. The weakest performance occurs in Scenario-B, where the *F1* is only 66.15% and the *Rec* significantly exceeds the *Pre*. Overall, GradBP (max) does not perform as well with unknown classes in open-set scenarios as it does with known classes.

GradBP (square root) reaches peak performance in Scenario-E, with an *F1* of 80.31%, a *Rec* of 63.28%, and a *Pre* of 94.10%. It degrades to its worst performance in open-set classification in Scenario-D, with an *F1* of 52.83%, which is 41.20% lower than our method, RoNeTC. Overall,

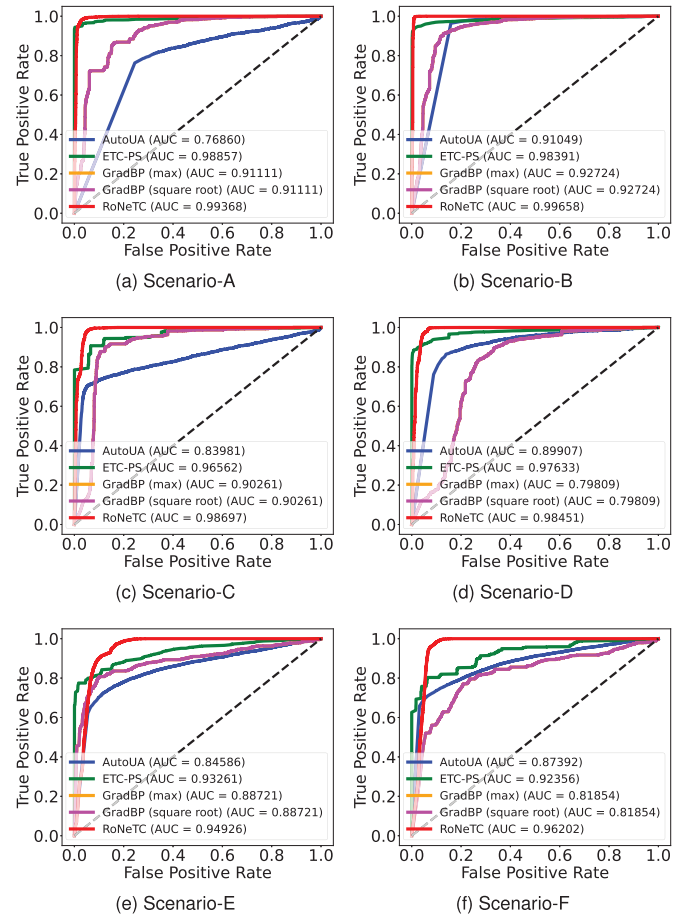


Fig. 8. Comparison of ROC curves and AUC values for the known and unknown classes in six open-set scenarios with the SOTA methods.

GradBP (max) consistently outperforms GradBP (square root) in open-set classification. Notably, GradBP (max) demonstrates consistently higher *Rec* than *Pre* across all scenarios, underscoring its strong ability to identify known classes. Conversely, GradBP (square root) exhibits greater *Pre* than *Rec*, indicating a more effective identification of unknown classes.

To clearly illustrate the open-set classification capability, we plot the ROC curves for both known and unknown classes and calculate the AUC values, as shown in Fig. 8. RoNeTC consistently achieves the highest AUC values across various scenarios, performing best in Scenario-B with an AUC value of 99.66%. Its ROC curve is closest to the upper left corner, indicating a low false positive rate and a very high true positive rate. This demonstrates that the classifier can accurately classify known classes while rarely misclassifying unknown classes as known class flows, correctly identifying almost all unknown samples in this scenario. In contrast, all other methods lie below the RoNeTC curve as the false positive rate changes, particularly AutoUA and GradBP (square root), which show a lower true positive rate at the same false positive rate. This indicates that their classification performance for known and unknown classes is inferior to RoNeTC.



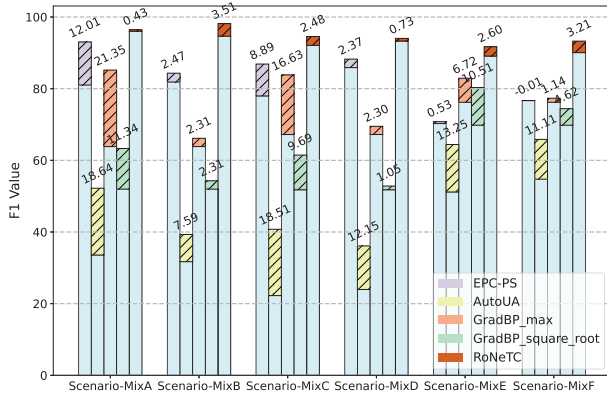


Fig. 9. Comparison of  $F1$  before and after adding a large number of unknown classes in six open-set scenarios with the SOTA methods. The colored sections with stripes indicate the magnitude of the decrease in  $F1$  after addition.

TABLE X  
OPEN-SET SCENARIOS CHANGES FOR ROBUSTNESS ANALYSES

| Scenario      | Known Classes | Sample Size | Unknown Classes          | Sample Size |
|---------------|---------------|-------------|--------------------------|-------------|
| Scenario-MixA | Dataset-I     | 12 200      | Dataset-I + Dataset-III  | 53 079      |
| Scenario-MixB |               |             | Dataset-III + Dataset-II |             |
| Scenario-MixC | Dataset-II    | 9000        | Dataset-I + Dataset-III  | 56 279      |
| Scenario-MixD |               |             | Dataset-III + Dataset-I  |             |
| Scenario-MixE | Dataset-III   | 44 079      | Dataset-II + Dataset-I   | 21 200      |
| Scenario-MixF |               |             | Dataset-I + Dataset-II   |             |

The red dataset serves as the additional added unknown samples set.

### C. Robustness Analysis

In our open-set Scenario-A and B, C and D, and E and F, each pair shares the same known training dataset, with the only difference being the unknown class set. We aim to assess the robustness performance of each open-set classification method by adding additional unknown sample classes while maintaining the original known samples, parameters, and thresholds unchanged. The experimental setup is detailed in Table X. Initially, each scenario involves one dataset for known classes and another for unknown samples. We expanded this setup by adding an additional dataset as unknown samples set without adjusting any parameters, including thresholds. The results depicted in Fig. 9 reveal noteworthy fluctuations in the  $F1$  across the majority of open-set classification methods. For instance, AutoUA exhibits a substantial decrease in  $F1$  across all scenarios upon the inclusion of unknown classes. Meanwhile, ETC-PS, recognized as one of the premier classification comparison methods overall, experiences a significant decline specifically in Scenario-MixA and MixC. In contrast, our proposed method, RoNetC, demonstrates only minor fluctuations. Although there is a slight 3% downturn in  $F1$  in Scenario-MixB and MixF, it still achieves the highest  $F1$  overall. This outcome highlights RoNetC's superior classification performance and excellent robustness when the number of unknown classes is significantly increased.

### D. Interpretability of Our Proposed Method

To explore what features the model has learned from network traffic, in this subsection, we use an open-source tool, "Lime" [35], which can explain the predictions of any classifier, to help us understand the prediction results of RoNetC. The results on Dataset-I are shown in Fig. 10 and

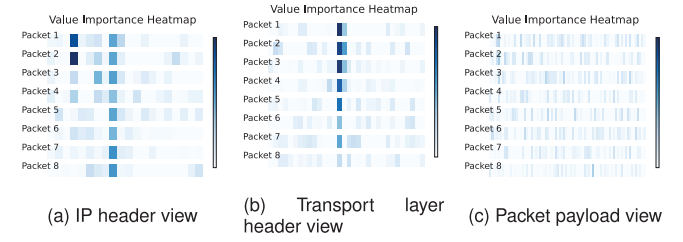


Fig. 10. Heat map of field importance for TLS flow that does not include the Handshake process.

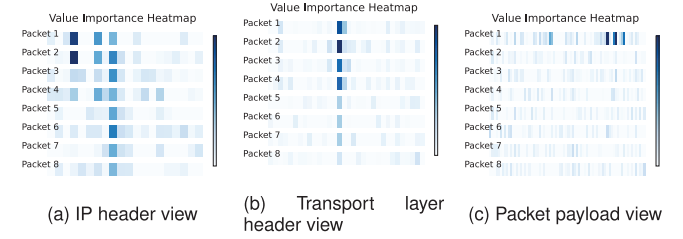


Fig. 11. Heat map of the field importance of TLS flow that contains the Handshake process.

Fig. 11, highlighting the heat maps of the three views of encrypted flows. Fig. 10 represents a TLS flow without the handshake process, while Fig. 11 represents a TLS flow with the handshake process. In both figures, it can be observed that in the IP header and Transport layer header views, certain fields within a single packet, as well as their corresponding positions across packets, are critical for classification, such as the "Total Length" and "Time to Live" in the IP header view at their important positions, and the "Window Size" in the Transport layer header view. Since these positions are fixed across packets, we can effectively model the global features of the flow by capturing the state transition relationships between different packets. In contrast, the payload view shows that, due to encryption, the importance of these fields for classification is significantly reduced. This is particularly evident in Fig. 10, which does not include the handshake process. However, for Fig. 11, as it includes the handshake process, the first packet is in plaintext, allowing extraction of information that contributes significantly to classification.

### E. Discussions

Currently, network traffic classification in open-set scenarios relies on a fixed number of intercepted packets and bytes for feature extraction. To handle flows that do not meet this fixed number, padding with zeros is typically applied. However, in real-world scenarios, the number of packets and bytes in network traffic fluctuates. This fixed-length feature extraction method lacks flexibility and often leads to wasted storage and computational resources.

Especially for real-time classification, easily identifiable classes might be accurately classified with only the first few packets and a small number of bytes. However, to maintain uniform input for the model, it is often necessary to wait for the capture of additional packets. This significantly increases processing time and reduces classification efficiency. There-

TABLE XI  
COMPUTATIONAL COMPLEXITY OF RoNETC  
IN THE CLASSIFICATION PHASE

| (a) Acronyms and Their Meanings |                                                 |
|---------------------------------|-------------------------------------------------|
| Acronym                         | Detail Meaning                                  |
| $H$                             | Height of the input tensor                      |
| $W$                             | Width of the input tensor                       |
| $C$                             | Number of channels in the input tensor          |
| $p$                             | Patch size                                      |
| $d$                             | Feature dimension within the Transformer module |
| $f$                             | Internal dimension of the Feed-Forward Network  |
| $H_{\text{head}}$               | Number of heads in the Transformer              |
| $N$                             | Number of Transformer blocks                    |
| $K$                             | Number of classes                               |

(b) Computational Complexity of the Classification Phase

| Core Component                 | Computational Complexity                                                                                            |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Global-Local Feature Extractor | $O\left(\frac{H \cdot W}{p^2} \cdot (C \cdot d + N \cdot d \cdot (H_{\text{head}} + d) + N \cdot d \cdot f)\right)$ |
| Dirichlet Distribution         | $O(K^2)$                                                                                                            |

fore, dynamically processing feature extraction could become a promising research direction for network traffic classification in open-set scenarios.

### F. Computational Complexity

We present the computational complexity of the core components during the classification phase, namely the global-local feature extractor and the Dirichlet distribution. Specifically, we demonstrate the computational complexity of these core components for a single view during the classification phase. The results are presented in Table XI, where Table XI(a) provides the meanings of acronyms, and Table XI(b) presents the computational complexity details. Since this is the classification phase, loss calculation is unnecessary, and thus KL divergence is excluded from the computational complexity.

From Table XI(b), it can be seen that the computational complexity of the Dirichlet distribution is  $O(K^2)$ , where  $K$  represents the number of traffic classes. The computational complexity of the Dirichlet distribution is related to the number of classes, whereas the computational complexity of the global-local feature extractor is independent of the number of classes. Moreover, we have the following relationship:

$$O\left(\frac{H \cdot W}{p^2} \cdot (C \cdot d + N \cdot d \cdot (H_{\text{head}} + d) + N \cdot d \cdot f)\right) \gg O(K^2) \quad (13)$$

Therefore, the computational cost of our method is less affected by the number of classes.

Additionally, we perform a quantitative analysis. We analyzed the time consumption of RoNeTC's classification phase in six open-set scenarios. The details of the scenario settings are presented in Table XII(a), and the time consumption results are shown in Table XII(b). The experiments are conducted on an A800 GPU paired with a 14 vCPU Intel(R) Xeon(R) Gold 6348 CPU @ 2.60GHz. For each of the six open-set scenarios, RoNeTC is run 20 times during the testing phase, and the average processing time is recorded.

The results indicate that RoNeTC achieves the highest processing speed and throughput in Scenario-A, reaching 220,361 flows/second. Across the other scenarios, there are minimal

TABLE XII  
TIME CONSUMPTION OF RoNETC IN THE CLASSIFICATION PHASE

(a) Scenario Setting

| Scenario            | Number of classes | Number of Packets | Number of Bytes | Batch Size | Number of GPU  | CPU                                              |
|---------------------|-------------------|-------------------|-----------------|------------|----------------|--------------------------------------------------|
| Scenario-A          | 13                | 8                 | 128             | 1024       | 1 × A800(80GB) | 14 vCPU Intel(R) Xeon(R) Gold 6348 CPU @ 2.60GHz |
| Scenario- $\bar{A}$ | 13                | 8                 | 128             | 256        |                |                                                  |
| Scenario-B          | 13                | 16                | 256             | 256        |                |                                                  |
| Scenario-C          | 19                | 4                 | 256             | 256        |                |                                                  |
| Scenario-D          | 19                | 16                | 256             | 256        |                |                                                  |
| Scenario-E          | 48                | 16                | 256             | 256        |                |                                                  |
| Scenario-F          | 48                | 4                 | 256             | 256        |                |                                                  |

Scenario- $\bar{A}$  and Scenario-A differ only in "Batch Size".

(b) Time Consumption of the Classification Phase

| Scenario            | Feature Extractor Time       | Dirichlet Distribution Time  | Total Time                    | Throughput     |
|---------------------|------------------------------|------------------------------|-------------------------------|----------------|
| Scenario-A          | $3.95 \times 10^{-5}$ s/flow | $5.88 \times 10^{-7}$ s/flow | $4.538 \times 10^{-5}$ s/flow | 220361 flows/s |
| Scenario- $\bar{A}$ | $1.91 \times 10^{-5}$ s/flow | $3.12 \times 10^{-6}$ s/flow | $2.22 \times 10^{-5}$ s/flow  | 45045 flows/s  |
| Scenario-B          | $1.97 \times 10^{-5}$ s/flow | $3.53 \times 10^{-6}$ s/flow | $2.32 \times 10^{-5}$ s/flow  | 43103 flows/s  |
| Scenario-C          | $1.98 \times 10^{-5}$ s/flow | $3.54 \times 10^{-6}$ s/flow | $2.33 \times 10^{-5}$ s/flow  | 42918 flows/s  |
| Scenario-D          | $1.92 \times 10^{-5}$ s/flow | $3.44 \times 10^{-6}$ s/flow | $2.26 \times 10^{-5}$ s/flow  | 44247 flows/s  |
| Scenario-E          | $1.97 \times 10^{-5}$ s/flow | $3.43 \times 10^{-6}$ s/flow | $2.31 \times 10^{-5}$ s/flow  | 43290 flows/s  |
| Scenario-F          | $1.96 \times 10^{-5}$ s/flow | $3.39 \times 10^{-6}$ s/flow | $2.30 \times 10^{-5}$ s/flow  | 43478 flows/s  |

s/flow and flows/s, where "s" represents seconds.

s/flow and flows/s, where "s" represents seconds.

differences in terms of "Feature Extractor Time", "Dirichlet Distribution Time", "Total Time", and "Throughput". For example, in Scenario-E(48 classes, 16 packets, with both batch size and byte count set to 256), the "Feature Extractor Time" is  $1.97 \times 10^{-5}$  s/flow, the "Dirichlet Distribution Time" is  $3.43 \times 10^{-6}$  s/flow, the "Total Time" is  $2.31 \times 10^{-5}$  s/flow, and the "Throughput" is 43290 flows/s. In contrast, in Scenario-B(13 classes, 16 packets, with both batch size and byte count set to 256), the "Feature Extractor Time" is  $1.97 \times 10^{-5}$  s/flow, the "Dirichlet Distribution Time" is  $3.53 \times 10^{-6}$  s/flow, the "Total Time" is  $2.32 \times 10^{-5}$  s/flow, and the "Throughput" is 43103 flows/s.

Notably, when the batch size in Scenario-A is set to the same as other scenarios (denoted as Scenario-Ā, with a batch size of 256), the differences in "Feature Extractor Time", "Dirichlet Distribution Time", "Total Time", and "Throughput" are similarly negligible compared to the other scenarios. These findings demonstrate that the time cost of RoNeTC is less influenced by the number of classes and more significantly affected by the batch size. In scenarios with a large number of samples, increasing the batch size can enhance processing efficiency. Alternatively, multi-GPU parallel processing can also be considered.

## VII. CONCLUSION

In this paper, we propose a novel and reliable open-set network traffic classification method, RoNeTC. The method is based on uncertainty estimation to quantify the reliability of the classification results and to demonstrate the decision reliability of open-set flow classification in a probabilistic manner, achieving robust and reliable performance. It combines multiple views by dividing a packet into three segments and extracting both global and local features from across and within packets from each view respectively, making full use of the prior knowledge of the known class samples to improve the accuracy rate. We perform uncertainty estimation by designing second-order classification probabilities and assess the reliability of classification decisions by constructing a distribution of classification probability distributions based on the classification evidence for each view. Finally, the classification decisions from multiple views are dynamically fused through the Dempster combination rule. We extensively evaluate RoNeTC on six open-set scenarios consisting of three datasets, as well as adding a large number of unknown samples

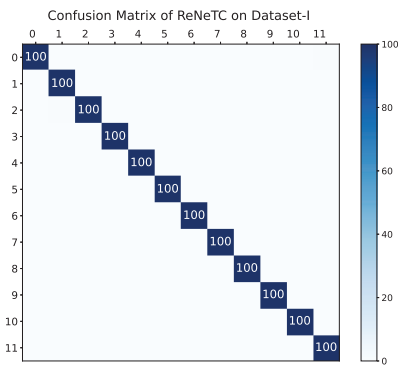


Fig. 12. Confusion Matrix of ReNeTC on *Dataset-I*.

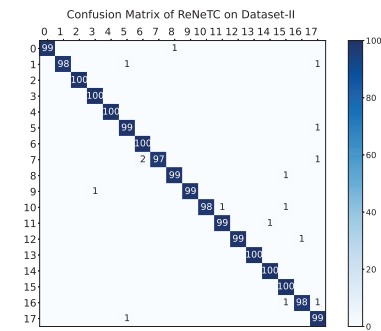


Fig. 13. Confusion Matrix of ReNeTC on *Dataset-II*.

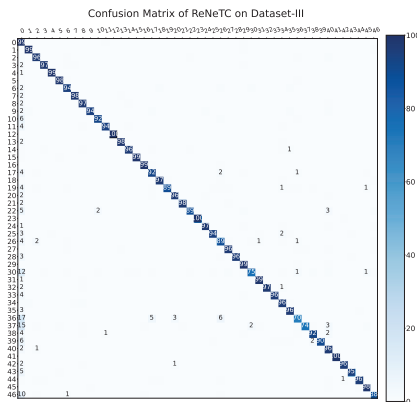


Fig. 14. Confusion Matrix of ReNeTC on *Dataset-III*.

for robustness comparisons. The experimental results show that RoNeTC is capable of handling open-set network traffic classification with superior robustness and reliability.

## APPENDIX

The confusion matrices of RoNeTC on the three closed-set datasets are shown in Fig. 12, Fig. 13, and Fig. 14.

## REFERENCES

- [1] L. Yang, A. Finamore, F. Jun, and D. Rossi, “Deep learning and zero-day traffic classification: Lessons learned from a commercial-grade dataset,” *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4103–4118, Dec. 2021.
- [2] R. Li, X. Xiao, S. Ni, H. Zheng, and S. Xia, “Byte segment neural network for network traffic classification,” in *Proc. IEEE/ACM Int. Symp. Quality Service*, Sep. 2018, pp. 1–10.

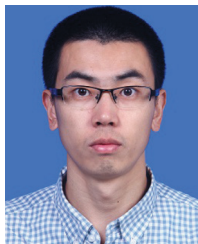
- [3] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, “MIMETIC: Mobile encrypted traffic classification using multimodal deep learning,” *Comput. Netw.*, vol. 165, Dec. 2019, Art. no. 106944.
- [4] J. Zhang, F. Li, F. Ye, and H. Wu, “Autonomous unknown-application filtering and labeling for DL-based traffic classifier update,” in *Proc. IEEE Conf. Comput. Commun.*, Oct. 2020, pp. 397–405.
- [5] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, and J. Weng, “Seeing traffic paths: Encrypted traffic classification with path signature features,” *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2166–2181, 2022.
- [6] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, “Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges,” *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 2, pp. 445–458, Jun. 2019.
- [7] R. F. Bismukhamedov and A. F. Nadeev, “Lightweight machine learning classifiers of IoT traffic flows,” in *Proc. Syst. Signal Synchronization, Generating Process. Telecommun. (SYNCHROINFO)*, Jul. 2019, pp. 1–5.
- [8] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, “FS-Net: A flow sequence network for encrypted traffic classification,” in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Aug. 2019, pp. 1171–1179.
- [9] C. Liu, Z. Cao, G. Xiong, G. Gou, S.-M. Yiu, and L. He, “MaMPF: Encrypted traffic classification based on multi-attribute Markov probability fingerprints,” in *Proc. IEEE/ACM 26th Int. Symp. Quality Service (IWQoS)*, Jun. 2018, pp. 1–10.
- [10] Y. Wang, H. He, Y. Lai, and A. X. Liu, “A two-phase approach to fast and accurate classification of encrypted traffic,” *IEEE/ACM Trans. Netw.*, vol. 31, no. 3, pp. 1071–1086, Jun. 2023.
- [11] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, “WF-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2024.
- [12] J. Van Amersfoort, L. Smith, Y. W. Teh, and Y. Gal, “Uncertainty estimation using a single deep deterministic neural network,” in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 9690–9700.
- [13] J. Moon, J. Kim, Y. Shin, and S. Hwang, “Confidence-aware learning for deep neural networks,” in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 7034–7044.
- [14] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, “DISTILLER: Encrypted traffic classification via multimodal multitask deep learning,” *J. Netw. Comput. Appl.*, vols. 183–184, Jun. 2021, Art. no. 102985.
- [15] A. Nascita, A. Montieri, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, “XAI meets mobile traffic classification: Understanding and improving multimodal deep learning architectures,” *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4225–4246, Dec. 2021.
- [16] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, “EBSNN: Extended byte segment neural network for network traffic classification,” *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 5, pp. 3521–3538, Sep. 2022.
- [17] Y. Wang, X. Yun, Y. Zhang, C. Zhao, and X. Liu, “A multi-scale feature attention approach to network traffic classification and its model explanation,” *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 2, pp. 875–889, Jun. 2022.
- [18] G. Bovenzi et al., “Benchmarking class incremental learning in deep learning traffic classification,” *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 1, pp. 51–69, Feb. 2023.
- [19] A. Nascita, A. Montieri, G. Aceto, D. Ciuonzo, V. Persico, and A. Pescapé, “Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI,” *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 2, pp. 1267–1289, Jun. 2023.
- [20] J. Qu et al., “An input-agnostic hierarchical deep learning framework for traffic fingerprinting,” in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 589–606.
- [21] Y. Chen and Y. Wang, “MPAF: Encrypted traffic classification with multi-phase attribute fingerprint,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7091–7105, 2024.
- [22] Y. Qian et al., “Enhancing resilience in website fingerprinting: Novel adversary strategies for noisy traffic environments,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7216–7231, 2024.
- [23] G. Aceto, D. Ciuonzo, A. Montieri, V. Persico, and A. Pescapé, “AI-powered internet traffic classification: Past, present, and future,” *IEEE Commun. Mag.*, vol. 62, no. 9, pp. 168–175, Sep. 2024.
- [24] S. Mehta and M. Rastegari, “MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer,” 2021, *arXiv:2110.02178*.
- [25] A. Dosovitskiy et al., “An image is worth 16×16 words: Transformers for image recognition at scale,” 2020, *arXiv:2010.11929*.
- [26] A. P. Dempster, “A generalization of Bayesian inference,” *J. Roy. Stat. Soc. B. Methodol.*, vol. 30, no. 2, pp. 205–232, 1968.



- [27] M. Sensoy, L. Kaplan, and M. Kandemir, "Evidential deep learning to quantify classification uncertainty," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, Jan. 2018, pp. 1–11.
- [28] B. A. Frigiyk, A. Kapila, and M. R. Gupta, "Introduction to the Dirichlet distribution and related processes," Dept. Elect. Eng., Univ. Washington, Washington, DC, USA, Tech. Rep. UWEETR-2010-0006, 2010, vol. 6, pp. 1–27.
- [29] K. Sentz and S. Ferson, "Combination of evidence in Dempster–Shafer theory," Sandia Nat. Lab., CA, USA, Tech. Rep. SAND 2002-0835, 2002.
- [30] A. Jsang, *Subjective Logic: A Formalism for Reasoning Under Uncertainty*. Cham, Switzerland: Springer, 2018.
- [31] N. J. Perkins and E. F. Schisterman, "The youden index and the optimal cut-point corrected for measurement error," *Biometrical J., J. Math. Methods Biosci.*, vol. 47, no. 4, pp. 428–441, Jun. 2005.
- [32] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.
- [33] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multidimensional IoT profiling dataset," in *Proc. 19th Annu. Int. Conf. Privacy, Security Trust (PST)*, 2022, pp. 1–11.
- [34] T.-D. Pham, T.-L. Ho, T. Truong-Huu, T.-D. Cao, and H.-L. Truong, "MAppGraph: Mobile-app classification on encrypted network traffic using deep graph convolution neural networks," in *Proc. Annu. Comput. Security Appl. Conf.*, 2021, pp. 1025–1038.
- [35] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you? Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.



**Xueman Wang** is currently pursuing the Ph.D. degree in computer technology with the College of Computer Science, Beijing University of Technology, Beijing, China. Her current research interests include network traffic classification and artificial intelligence.



**Yipeng Wang** (Senior Member, IEEE) received the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences (CAS), China, in 2014. He is currently with the College of Computer Science, Beijing University of Technology, China. He has published more than 50 research papers in refereed international journals and conferences, such as IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE/ACM TRANSACTIONS ON NETWORKING, the IEEE International Conference on Network Protocols, and IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT. His research interests include networking, network security, and machine learning. He was a recipient of the Best Paper Award at the IEEE International Conference on Network Protocols (ICNP) for his protocol format inference technology. He serves as a Regular Reviewer for IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, and IEEE INTERNET OF THINGS JOURNAL. He serves as a Program Committee of IJCAI-21, IJCAI-22, IJCAI-23, IJCAI-24, and IJCAI-25.



and Technology.

**Yingxu Lai** received the Ph.D. degree from Chinese Academy of Sciences in 2003. She joined the College of Computer Science, Beijing University of Technology, in 2003. She was a Visiting Scholar with Arizona State University from 2013 to 2014. She is currently a Full Professor. She has had over 70 papers published in various international journals and conferences. Her research interests include cloud computing, network security, edge computing and trusted computing. She is an Associate Editor of *Journal of Artificial Intelligence*



**Zhiyu Hao** received the Ph.D. degree in computer system architecture from Harbin Institute of Technology in 2007. He is currently a Professor with the Zhongguancun Laboratory, Beijing. He has published over 40 papers in journals and conferences, including IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, and IEEE TRANSACTIONS ON SERVICES COMPUTING. His research interests include network security and system virtualization.



**Alex X. Liu** (Fellow, IEEE) received the Ph.D. degree in computer science from The University of Texas at Austin in 2006. He is currently the President of the Software Engineering Institute and the Chief Security Officer of Midea Group. Before that, he was the Chief Scientist of Ant Group and further before that, he was a Professor with the Department of Computer Science and Engineering, Michigan State University. His research interests include cybersecurity, cloud computing, dependable computing, and privacy-preserving computing. He

is a member of Academia Europaea, a member of European Academy of Sciences and Art, an IET Fellow, an AAIA Fellow, and an ACM Distinguished Scientist. He received the IEEE & IFIP William C. Carter Award in 2004, the National Science Foundation CAREER Award in 2009, Michigan State University Withrow Distinguished Scholar (Junior) Award in 2011, and Michigan State University Withrow Distinguished Scholar (Senior) Award in 2019. He received the Best Paper Awards from SECON-2018, ICNP-2012, SRDS-2012, and LISA-2010. He has served as the TPC Co-Chair for ICNP 2014 and IFIP Networking 2019. He has served as an Editor for IEEE/ACM TRANSACTIONS ON NETWORKING and an Area Editor for *Computer Communications*. He is also an Associate Editor of IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING and IEEE TRANSACTIONS ON MOBILE COMPUTING.